12.010 Homework #1 **Due Thursday September 29, 2011**

Question (1): (10-points) Express the following numbers in base 2, 8, 10, and 16 as appropriate (subscript denotes the base of the input number). (See notes on web page and power point)

65261_{10}

15_{16}

5655_{8}

6013_{8}

Question (2): (10-points) How long will it take on a 56K modem to transfer a 56 Mbytes file? How long on 45 Mbps T1 Ethernet line? Calculation should be accurate to 3-significant digits.

Question (3): (10-points) In a computer with 1 Gbytes of memory, what is the maximum size matrix that can be stored with 8-bytes per number in (a) full storage i.e., NxN, (b) lower triangular form. What are the values if the numbers are stored in 4-byte numbers (assume all of the memory can be used for storage)? **The numbers here should be exact, not approximations.**

Question (4): (10-points) In class we gave the precision and range for IEEE 4-byte floating point numbers. What is the precision and range for IEEE 8-byte floating-point numbers? (IEEE 8-byte floating point uses 11 bits for the exponent and 53 bits for the mantissa (don't forget about the sign bits). (see Notes on web page and power point)

Question (5): (10-points) Develop an algorithm to sum the squares of integers between two values n and m i.e. $\sum_{i=n}^{m} i^2$**.** The solution is not computer code but rather the method to be used written out in English sentences.

Question (6): (50-points) Design an algorithm to predict how long it will take a bicyclist to travel a specified course given the characteristics of the bicycle and rider and the amount of power the bicyclist can output as a function of time. This problem will lead to later questions where you will write a program to simulate the motion of the bicyclist. In this question you are not writing computer code: You are finding the equations you will need to use and thinking about how to implement those equations into an algorithm to solve this problem.

You will write (in English) a description of
(1) Equations of motions of the bicycle. Concentrate here on basic energy and force equations. (Final problems will be 2-D motion i.e., no corners)
(2) Converting forces and accelerations: Given the forces acting on the bicycle, how do you calculate the motion of the bicycle and the power requirements.

(3) Input and outputs for the program. What types of information will the program need for input and what might it output?
(4) List a set of methods that you will use to validate that your program outputs the correct values and how you will judge “correct”.
(5) Given the nature of this problem, describe what you think will be the issues that you will need to consider carefully when programming this problem.

(Your answer should address each of the items above. Your answer should be equations and written description.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #1 **Due Thursday September 29, 2011**

Solutions

Question (1): (10-points) Express the following numbers in base 2, 8, 10, and 16 as appropriate (subscript denotes the base of the input number). (See notes on web page and power point)

65261_{10}

15_{16}

5655_{8}

6013_{8}

Answer: Since each of the columns in a number represents the multiplier of the base to power of the (column –1), we can solve this problem by direct calculation. The easiest problems of this type are conversion to base10 since this is the system most of us are familiar with. The conversion from base10 to some other is the most difficult, however in a question such as this (conversion to multiple systems), having done one the rest should be easy. One technique, I will show below for the second question—the conversion of 65261_{10}. In base 16, we know that the number will look like

$$e'16^4+d'16^3+c'16^2+b'16^1+a'16^0 = e'65536+d'4096+c'256+b'16+a.$$

Since our value is less than 65536($=16^4$), we know that e and above must zero. To compute d, we divide the number by 4096 and take the integer part (=d), then subtract d´4096, from the original number and obtain a remainder (d=15 which is Hex F, the remainder R=3821). We repeat the same process on the remainder, this time dividing 256 to obtain c. This results in c=14 which hexadecimal digit E. The remainder is divided by 16 to obtain b, and the final remainder a. To compute the other terms, we convert the hexadecimal to binary, and convert the binary. To do this, we use the fact that a 4-bit binary number can represent each digit in the hex number. The table below shows the concept

Hexadecimal	F				E				E				D			
Binary	1	1	1	1	1	1	1	0	1	1	1	0	1	1	0	1
Octal	1	7			7			3			5		5			
Notice how the binary digits can be grouped and mapped to the digits in the hexadecimal and octal numbers.																

Final answers to all the values are given below. Values in bold are the original numbers given.

Base 2	Base 8	Base 10	Base 16
1111111011101101	177355	**65621**	FEED
10101	25	21	**15**
101110101101	**5655**	2989	BAD
110000001011	**6013**	3083	C0B

Question (2): (10-points) How long will it take on a 56K modem to transfer a 56 Mbytes file? How long on 45 Mbps T1 Ethernet line? Calculation should be accurate to 3-significant digits.

Answer:

(a) There are two answers to this problem. Strictly kbs is 1024 bits-per-second and k bs is 1000 bits-per-seconds. However in common use today is kbs to mean either 1024 or 1000 bits-per-second. Hence a 56K modem could transfer data at 56*1024 bits-per-sec, and a 56 Mbyte file contains 56*1024*1024*8 bits, the transfer will take 8192 seconds, assuming that the full transfer rate is achieved, or the rate could be 56*1000bps, in which case the time will be 8388.6 seconds. In practice these modems rarely achieve rates higher than 45 kbps. A standard that seems to be adhered to is b means bit and B means byte.

(b) For a 45 Mbps T1 Ethernet line the transfer rate is 45*1024*1024 bits-per-second, and again assuming full transfer rate, it would take 9.96 seconds or if 1000 bps is assumed 10.44 seconds.

Question (3): (10-points) In a computer with 1 Gbytes of memory, what is the maximum size matrix that can be stored with 8-bytes per number in (a) full storage i.e., NxN, (b) lower triangular form. What are the values if the numbers are stored in 4-byte numbers (assume all of the memory can be used for storage)? **The numbers here should be exact, not approximations.**

Answer:

(a) In 1Gbytes, 1024*1024*1024 bytes of data can be saved. An NxN matrix stored in 8-byte floating point will require N*N*8 bytes of storage. Therefore, the size of matrix that can be stored is $N = \sqrt{1024*1024*1024/8} = 11585$ i.e., a 11585x11585 matrix (with 5503 real*8 bytes remaining).

(b) If lower triangle form is used, then the storage of the matrix requires Nx(N+1)/2 elements, each of 8 bytes, and therefore the maximum sized matrix is $N = \frac{-1 \pm \sqrt{1+8M}}{2}$ where M is the total memory in real*8 words (use only positive root). The exact positive solution to this equation is N=16383.5, but we must use the integer value for this. Therefore N=16383, leaving us with 65536 bytes to use for any program that manipulates this matrix.

If 4-byte floating point is used, then we can store twice the number of values but since the matrix size goes as N^2 the size of matrix will only go up by √2 for the full storage case and ~√2 for the lower diagonal case. Therefore for full storage, N=16384, and for lower triangular form, N=23169.

Question (4): (20-points)) In class we gave the precision and range for IEEE 4-byte floating point numbers. What is the precision and range for IEEE 8-byte floating-point numbers? (IEEE 8-byte floating point uses 11 bits for the exponent and 53 bits for the mantissa (don't forget about the sign bits). (see Notes on web page and power point)

Answer: Since 10 bits are allocated to the exponent; the largest number that can be represented is 2^{10}-1=1023. Therefore the total range of the exponent is $2^{1023}=9\acute{}10^{307}$ or approximately 10 to the powers of –308 to 308. In the mantissa we have 53 bits, leaving 52 bits to represent the value of the mantissa (one sign bit). The maximum number that can be represented in $2^{52}=5\acute{}10^{15}$, and hence we have approximately 15 significant digits.

Question (5): (10-points) Develop an algorithm to sum the squares of integers between two values n and m i.e. $\sum_{i=n}^{m} i^2$**.** The solution is not computer code but rather the method to be used written out in English sentences.

Answer: There are several ways of solving this problem in terms of algorithm design. We examined this type of problem in lecture 1 and there are two practical solutions to the problem.
(1) The easiest type algorithm is to directly compute the sum and so this would look like:
- Get the input values of n and m
- Initialize a variable that will contain the sum to zero
- Loop values of i between n and m
  Add to the sum the value if i*i
- End loop
- Output the sum.

(2) A faster method that requires more thought is to derive an expression for the sum that can be directly computed. This initially takes more of the programmer's time but will execute faster later. There are formulas for the sums of powers of integer and one example is
CRC Standard Mathematical Tables and Formulae, 30th edition, which is available on-line at
http://www.mathnetbase.com/ejournals/books/book_summary/summary.asp?id=614
http://www.mathnetbase.com/books/614/cr2479_01.pdf section 1.2.12, Page 21, Sums of powers of integers, gives the expression we need.
The sum of i2 from 1 to n is n(n+1)(2n+1)/6 and the formula we need is [m(m+1)(2m+1)-n(n+1)(2n+1)]/6. This algorithm will look like.
- Get the input values of n and m
- Set sum to [m(m+1)(2m+1)-n(n+1)(2n+1)]/6
- Output the sum.

Issues that can be encountered:

(a) We have seen that integers can only reach finite value: 4-byte integers are the most common and have a maximum value of 2147483647. When m=1860 and n=1, the final result is within the range limit, but the intermediate calculation of n(n+1)(2n+1) would exceed the range of integers. There are two types of solutions to this: (a) divide one the terms by 6 first and then multiple by the others (care is needed here in that integer calculations will round down unless the result is integer or (b) do the calculation in floating point can convert the final answer to the nearest integer (because there will be some rounding. Integer overflows are often a problem when factorials are used and these calculations are often better down in floating point.
(b) There can problems with the second method when m and n are close and large. If m=n+d and d is small compared to n, then the sum is approximately n^2 d. For d=1, n and m as large as 46340 can be computed without integer overflow. In these cases, the simple algorithm (1) would be fast and easy to implement. Some analytic manipulation of the expression in (2) could yield a closed form expression using just n and d that might be less prone to integer overflow. This will take more time to develop and errors could be made in the derivation. Sometime the straightforward "brute force" algorithm is the best.

Question (6): (50-points) Design an algorithm to predict how long it will take a bicyclist to travel a specified course given the characteristics of the bicycle and rider and the amount of power the bicyclist can output as a function of time. This problem will lead to later questions where you will write a program to simulate the motion of the bicyclist. In this question you are not writing computer code: You are finding the equations you will need to use and thinking about how to implement those equations into an algorithm to solve this problem.

You will write (in English) a description of
(1) Equations of motions of the bicycle. Concentrate here on basic energy and force equations. (Final problems will be 2-D motion i.e., no corners)
(2) Converting forces and accelerations: Given the forces acting on the bicycle, how do you calculate the motion of the bicycle and the power requirements.
(3) Input and outputs for the program. What types of information will the program need for input and what might it output?
(4) List a set of methods that you will use to validate that your program outputs the correct values and how you will judge "correct".
(5) Given the nature of this problem, describe what you think will be the issues that you will need to consider carefully when programming this problem.

(Your answer should address each of the items above. Your answer should be equations and written description.

Answer:

(1) Forces and equations of motion

This is a basic force balance analysis, which can be affected by many different processes. The basic motion must satisfy **F**=*m***a** where **F** is the (vector) sum of the forces acting on the bike, *m* is the mass of the rider and bike and **a** in the (vector) acceleration of the body. Consider the forces:

(a) Gravity: Since this is bike we can take it to be constant (i.e., no great height changes). In a very general solution we would treat gravity as height dependent.

(b) Centripetal force that will be used to keep the bike on the path to be followed. This force is need even when motion is just in 2-D (ie., height and horizontal distance).

(c) Drag: Normally drag will act along a direction opposite to the velocity vector of the body. On a bike there are two types: Wind drag which will depend on velocity squared, and rolling drag which just depends on the mass of the bike and rider and an effective friction coefficient.

(d) Force added by rider. For a bicycle this can complicated. The rider applies a force to the pedals and the force is applied over a distance as the pedal moves. The force times the distance traveled is the work done by the rider. This work results in a force at the tires on the ground and so we can think of the work being done as the force at the tires on the ground times the distance the bike moves. The complications here are that there is energy lose going from the pedals to tires. This lose could be quantified or we could just use the work at the tires (knowing then that this is minimum amount of work the rider will need to do).

The next important concept is that the rate of doing work is power and hence the power used by the rider will be

$$P_r = d(F_r D)/dt = F_r V_b \quad \Rightarrow F_r = P_r / V_b$$

Hence if we know the power supplied by the rider and the velocity of the bike, we can compute the force supplied by the rider the wheels. This force acts along the direction of the road surface. Notice here that a zero velocity, the force is infinite and this means at low velocities the rider cannot supply all the power possible because the force would be too large. Therefore we need to have an upper limit on the rider force at low velocities.

We need to worry about the accuracy of the calculation. This will affect the details of the numerical techniques that we use.

How to solve the problem? The basic choice is always between analytic solutions and numerical solutions. For steady state solutions over flat topography a force balance equation can be written in which the force supplied by the rider matches the drag and rolling forces. However, our problem is more complicated and so a numerical solution is needed. Note, however, we can partially test our code with the steady state solutions.

Numerical Solution: Basic equation of motion:

$$\ddot{\mathbf{x}}(t) = \mathbf{F}(t)/m$$

$$\dot{\mathbf{x}}(t) = \dot{\mathbf{x}}_o t + \int_o^t \ddot{\mathbf{x}}(s)ds$$

$$\mathbf{x}(t) = \mathbf{x}_o + \int_o^t \dot{\mathbf{x}}(u)du = \mathbf{x}_o + \int_o^t \left(\dot{\mathbf{x}}_o u + \int_o^u \ddot{\mathbf{x}}(s)ds \right) du$$

where **x** is the position and the dots above **x** denote derivatives, **F** is the vector sum of the forces applied to be body, and t, u and s are times.
There are two methods to solve the problem:

(a) Solve in 2-D with say x and z coordinate directions (i.e., x above is a 2-D vector) and include forces that keep the bike on the path. This approach is the only one that would work for a 3-D problem.
(b) Solve in 1-D by projecting all forces into the direction of travel and then assuming that normal forces balance and keep the bike on the path. This is basically the roller coaster solution and should give the same results as (1) if the bike does not become airborne. Solution (2) is easier but not as flexible as solution (1). It provides a means of testing solution (1).

Force calculations (equations that will be needed).
Gravity: **F** will be a vector pointing down which may depend on the height of the body (or could be constant for small displacements of the body). The formulas for gravity are

$$F = \frac{GMm}{r^2} = \frac{GMm}{(R+h)^2} \approx \frac{GMm}{R^2}(1 - 2h/R) = (9.806 - 3.0784 \times 10^{-6} h)m$$

where GM is the gravitational constant times the mass of the Earth (GM=3.98x10^{14} m^3/sec^{-2}), R is the radius of the Earth 6.3708x10^6 m, and h is the height of the bike above the surface. (The numerical values above are m/s^{-2} and h is in meters). Height dependence will only be needed for a very mountainous terrain. In our problem we can keep gravity constant at 9.8 m/s^2.

Centripetal Force: Curvature effects on the path followed by the bike require that the centripetal forces be included even when the trajectory is 2-D (i.e, along the path and up/down direction.

$$\boldsymbol{F}_c = -mV^2 \frac{d^2z}{dx^2}$$

where x and z are the horizontal and vertical coordinates and *V* is the velocity of the bike. Correctly accounting for the centripetal force will keep the bike on the specified path. If the curvature is too large in a convex sense (i.e., top of a hill) then the bike could leave the path. This case is one that needs to be considered when the solution to the problem is programmed.

Drag Force: The drag force is modeled by

$$\mathbf{F}_d = -\frac{1}{2} C_d \rho \mathbf{V}^2$$

where C_d is the drag coefficient again depending on wing area and aspect ratio, *r* is the air density, **V** is the velocity Drag acts along the wing shape.

Rolling Force:

$$\text{Rolling Drag} \quad F_r = -M_r g C_r$$

where Cr is a rolling coefficient and represents the effects of friction. It acts opposite to the direction of motion but its magnitude is constant.

The motion of the bike can be computed by vector summing all the forces, including the reaction force from the surface the bike sits on (unless the bike becomes airborne) and computing the accelerations. The accelerations are integrated to compute velocities and positions. The power supplied by the rider may be integrated in time to compute the total energy used (and thus the number of calories expended by the rider).

(2) Converting forces and motion

The forces are converted into accelerations by dividing by the mass of the bike and rider. The mass is needed when the energy used by the rider is computed. To compute the motion of the bike, the accelerations need to be integrated to yield velocities and velocities integrated to get position. The position along the track determines the slope and curvature of the path, which are needed to compute the accelerations. The integration is not simple time integration but rather the system is coupled. We need to solve a 2nd order differential equation.

Method for solving equations

Simple integration: Euler's method is simplest and least accurate method. In numerical integration, small time steps are taken from the initial conditions and at each step; the acceleration and current velocity are used to compute the velocity and value at the end of the time step. In our case, using the θ_1" and θ_2" second derivatives we can compute updated values of θ_1' and θ_2' and θ_1 and θ_2 after each time step. The so-called boundary conditions or initial conditions, give the initial values of θ_1' and θ_2' and θ_1 and θ_2 at time zero.

Mathematically Euler's method is written below where the y will be θ_1 or θ_2 and the function *f* is the expressions above and Δt is the time step. So here, the value of y and y' at time t_n is stepped to time t_{n+1}.

Solution to $y'' = f(t, y, y')$ for a step in t (time) of Δt

$$y_{n+1} = y_n + \Delta t[y'_n + \Delta t f(t_n, y_n, y'_n)/2] + O(\Delta t^2)$$

$$y'_{n+1} = y'_n + \Delta t f(t_n, y_n, y'_n)$$

The term $O(\Delta t^2)$ means the error is "of-order-of" Δt^2. This means that if the time step is halved, the error should reduce by $(1/2)^2$ or a quarter. Other methods have much better error performance.

Euler's method is very inaccurate and a better approach tries to take into account that y'' is changing over the time step Δt. The Runge-Kutta integration is one of the most common methods for improved performance. The equations below are a 4th order

Runge-Kutta algorithm given at
http://www.convertit.com/Go/Convertit/Reference/AMS55.ASP?Res=150&Page=897
equation 25.5.20 and is from Abramowitz and Stegun, Handbook of Mathematical functions. The function for the accelerations need to be called more frequently with this method but the accuracy is O(Δt^5) which means that if the time step is halved, the error should reduce by $(1/2)^5$ or 1/32. This method should run much faster than Euler's method because a much larger step size can be taken.

Solution to $y'' = f(t, y, y')$ for a step in t (time) of Δt

$$y_{n+1} = y_n + \Delta t[y'_n + \frac{1}{6}(k_1 + k_2 + k_3)] + O(\Delta t^5)$$

$$y'_{n+1} = y'_n + \frac{1}{6}[k_1 + 2k_2 + 2k_3 + k_4]$$

$$k_1 = \Delta t f(t_n, y_n, y'_n)$$

$$k_2 = \Delta t f(t_n + \Delta t/2, y_n + \Delta t y'_n/2 + \Delta t k_1/8, y'_n + k_1/2)$$

$$k_3 = \Delta t f(t_n + \Delta t/2, y_n + \Delta t y'_n/2 + \Delta t k_1/8, y'_n + k_2/2)$$

$$k_4 = \Delta t f(t_n + \Delta t, y_n + \Delta t y'_n + \Delta t k_3/2, y'_n + k_3)$$

There are equivalent Runge-Kutta formulas when first derivatives are given such as in the second set of equations above that use the momenta.

(3) Input/output for program.

The user inputs we will need are:

(a) Information about the path followed by bike. There are many ways this information could be entered such as heights along the path or simple functions. Because we need to compute the curvature, constructing the surface as the sum of a set of sine and cosine functions of different wavelengths is an appealing input method. The type of expansion is called a Fourier series. In theory any surface can be represented by a Fourier series and so the coefficients and wavelengths could be entered. Alternatively, points along the path could be entered and the Fourier coefficients computed in the program.

(b) Properties of bike and rider: Mass, Rolling and Drag coefficients. Power use profile. This input is more tricky since it is not clear how to parameterize the use of power by the rider. For example will the power from the rider decay with time? And if it does decay, how would the decay be represented? In the homework codes, we will assume constant power use by the rider.

(c) Accuracy needed for the integration. This is critical because the accuracy requirement will set the characteristics of the integration especially the step size.

Output could simply be writing results to a screen or file, or we might put an interface to graphics routines that would plot the results. The specific output depends on the user

requirement. Types of output could position and velocity as a function of time and the total energy used by the rider.

(4) Methods to validate program.

As mentioned above there are two ways to solve this problem one which is more limited in its validity conditions (i.e., curvature and speed can not be too great) but the two implementations could be used to validate each other in the domains where they are both valid.

Individual parts of the code can be checked e.g. is the drag force reasonable, does the bike accelerate downhill even with no rider power? Does it reach terminal velocity going hill and does it stop going up-hill when no new power added.

Some conditions such a flat surface and constant speed can be solved analytically to validate the solution. Care needs to be exercised here because not all cases are tested with this validation. For example, curvature effects are not tested with the simple analytical solutions.

Rider power could be cut to zero at some, time and the decay of the speed could be verified.

Knowing how much power riders can supply can be used to test the order of magnitude correctness and the calories used riding a bike.

(5) Issues in programing the solution.

There are issues common to all program of this type such as how maintain and achieve the accuracy requested by the user especially if the requested accuracy cannot be achieved. These issues determine the integration method and the step size used in the integration.

Specific to the bike some issues are:

(a) What happens in the program if hills are too steep and the bike can't get up them?

(b) What happens at the tops of hills if the bike leaves the ground? Do we want the program to handle this case? (The 1-D solution method does not allow this to happen).

(c) What methods should be used to compute energy used? Do we compute as the equations are solved or save the results of the integration and then compute the energy?

Example layout of a program for this problem:

Here we write out in English the major parts of the program (basically what the main program and each major module will do).

Main program:

Start: Tell user what the program does and

Get_input — Get input from user either by reading from keyboard or file, or by decoding command line. Input will include surface description, coefficients for forces, mass, and power use profile.
Initialize the integrator and time, and pre-calculate any quantities that are constant (for example value of gravity)
Loop over the time needed starting at zero and going to the time requested by the user in time steps requested by the user.

Determine the number of steps needed by the integrator. Basically do this by using the last step size, and see if the accuracy is adequate. If it is not then we half the step size and take twice as many steps. If the accuracy if far better than we need, then double step size and see if still OK. By using multiples of two, we can always fit an integer number of steps into time step requested by the user.

Integrate to the next time step

Output the results

End loop over time steps
Output any final values such as total energy used and travel time.
End Main

Get_input routine
This will basically collect the user input that we need. These include:

(a) Information about the path followed by bike. In theory any surface can be represented by Fourier series and so these coefficients and wavelengths could be entered.
(b) Properties of bike and rider: Mass, Rolling and Drag coefficients. Power use profile. This is more tricky since it is not clear how to parameterize the use. In homework codes, we will assume constant power use (except for graduate homework).
(c) Accuracy needed for the integration

Step size calculation:
The issue here is getting the step size that satisfies the accuracy requirements. To get the error estimate to decide on step size we could try two step sizes and check the difference in the results. If the difference is small then the current step size is OK.

Integrate to the next time step: Given we have the step sizes this is now relatively easy. Note the routines to do this calculation, e.g., gravity, drag, rolling forces etc will be shared with the error/step size calculation modules.

Output of results: This could simply be writing results to a screen or file, or we might put an interface to graphics routines that would plot the results. Again depends on user requirement.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #2 **Due Thursday, October 20, 2011**

Question (1): (25-points) (a) Write, compile and run a fortran program which generates a table of error function (erf) and its derivatives for real arguments (*z*) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the program and give an example of the output.
(b) How would you change this program if 10 significant digits were required?
Fortran source code should also be supplied

Question (2): (25-points).
Write a program that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The program should not be specific to the lengths of your name (ie., the program should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.
Hints:
Look at the ASCII table and check the relationship between upper and lower case letters
Intrinsic function CHAR and ICHAR convert between character strings and ascii codes and visa versa.
Reading with a * format will allow three strings to be read on the same line i.e.,

```
read(*,*) string1, string2, string3
```

will allow all names to be on the one line.
Writing with a format of a single a (instead of a10 for example) will output only the number of characters in the string to be output. To avoid extra spaces, only print the number of characters needed using the 1:N feature where N is the number of characters needed.

Question (3): (50-points) Write a Fortran program that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where *H(x)* is the height of the path above the starting height, *S* is a slope in m/m, *A* and *B* are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and *V* is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, *g* is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and *V* is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your program can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The program can assume a constant power scenario and constant force at low velocities.

As a test of your program use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.

(b) The position and velocity of the bike tabulated at a 100-second interval.

Assume the following values

C_d = 0.9

C_r = 0.007

A_r = 0.67 m^2

ρ = 1.226 kg/m^3

g = 9.8 m/s^2

M_r = 80 kg

Your answer to this question should include:

(a) The algorithms used and the design of your program

(b) The Fortran program source code (I will compile and run your programs).

(c) The results from the test case above.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #2 Due Thursday, October 20, 2011

Question (1): *(25-points) (a) Write, compile and run a fortran program which generates a table of error function (erf) and its derivatives for real arguments (z) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).*

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the program and give an example of the output.
(b) How would you change this program if 10 significant digits were required?
Fortran source code should also be supplied

The main decision to be made here is how to implement the evaluation of the error function, Erf. The derivative is a simple expression and poses no challengers. Additional considerations are the formatting of the table but this is relatively easy in Fortran.

The http://mathworld.wolfram.com/Erf.html gibes the Maclaurin series for evaluating the Erf function

$$\operatorname{Erf}(z) = \frac{2}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(-1)^n z^{2n+1}}{n!(2n+1)}$$

This formula will work for all values of the argument z (which can be complex as well). To implement this equation we note the sum is to infinity, which will take a long time to compute. So we need to decide on the number of terms to include. As we look at the expression, for |z|<1 the series is well behaved in that as n goes to infinity, z^{2n+1} will go to zero (i.e., the terms get smaller and smaller). However for |z|>1, z^{2n+1} will go to infinity as n goes to infinity. These ever increasing values are reduced in size by the n! in the denominator. The magnitude of n! could cause problems. n! grows rapidly and will lead to over flow of integer*4 size limits when enough terms in the series. Sample calculations of the series suggest at for z-3.0, n should be 32 to obtain 10^{-6} accuracy in the calculation. Factorial(32) is 2.6×10^{35} which 25-orders of magnitude larger than the maximum value possible with integer*4. Therefore although factorial in an integer, it will need to be saved a real*8 in this calculation.

Another aspect of this series is the alternating sign $(-1)^n$. For arguments greater than 1, the series will be summing and subtracting numbers that are large and this can introduce rounding errors. (For example, for z=3, n=7 generates a contribution of 189.8

to the series. The terms for n=8 tends to cancel this term). Hence if large arguments are to be used, an alternative to the Maclaurin series should be considered. The Wolfram site gives some alternative expressions for large arguments.

In gfortran and gcc, there is an intrinsic erf function and if the user routine called erf is used, a warning is printed saying that there is a conflict between the user routine and the intrinsic routine. In gfortran, the intrinsic routine is used and not the user version which is rather surprising. (Normally, user routines over-ride system functions but since this an intrinsic function conflict this rule must not apply). On a unix system, man erf will tell you about the routine.

Code design:

The major issue with accuracy of the series calculations and factorial magnitude were discussed above. The major components of the code are to
(1) Output table headers so that the user knows what is being tabulated.
(2) Functions to evaluate erf and its derivative. In my implementation, the erf functions evaluates how many terms are needed to obtain 10^{-6} accuracy and so used different numbers of terms for each argument.
(3) Function needed to evaluate factorial. As noted above this is a real*8 function to ensure that the result does not overflow.

Solution:
The fortran code I used is linked to http://geoweb.mit.edu/~tah/12.010/HW02_01_11.f.
The output of my program is given below.

```
TAHComputer[288] gfortran HW02_01_11.f -o HW02_01
TAHComputer[289] HW02_01

-------------------------------------------------
TABLE OF ERROR FUNCTION (ERF) AND FIRST DERIVATIVE
-------------------------------------------------
| Argument   |     ERF    | d(ERF)/dx   |
|____________|____________|_____________|
| -3.000     | -0.99998   |  0.00014    |
| -2.750     | -0.99990   |  0.00059    |
| -2.500     | -0.99959   |  0.00218    |
| -2.250     | -0.99854   |  0.00714    |
| -2.000     | -0.99532   |  0.02067    |
| -1.750     | -0.98667   |  0.05277    |
| -1.500     | -0.96611   |  0.11893    |
| -1.250     | -0.92290   |  0.23652    |
| -1.000     | -0.84270   |  0.41511    |
| -0.750     | -0.71116   |  0.64293    |
| -0.500     | -0.52050   |  0.87878    |
| -0.250     | -0.27633   |  1.06001    |
|  0.000     |  0.00000   |  1.12838    |
|  0.250     |  0.27633   |  1.06001    |
|  0.500     |  0.52050   |  0.87878    |
|  0.750     |  0.71116   |  0.64293    |
|  1.000     |  0.84270   |  0.41511    |
```

1.250	0.92290	0.23652
1.500	0.96611	0.11893
1.750	0.98667	0.05277
2.000	0.99532	0.02067
2.250	0.99854	0.00714
2.500	0.99959	0.00218
2.750	0.99990	0.00059
3.000	0.99998	0.00014

(b) How would you change this program if 10 significant digits were required?
Several changes would be needed to output to 10-signifciant digits.

(1) The format would need to change for the extra digits. In particular F8.5 would need to be changed to F14.10
(2) The headings format would need to be changed to re-align the columns
(3) In the Erff function itself, the eps variable would be changed to 1.0d-11 to ensure the 10-digits of precision for the output.
(4) All variables would need to be real*8.

Question (2): (25-points).
Write a program that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The program should not be specific to the lengths of your name (ie., the program should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.
Hints:
Look at the ASCII table and check the relationship between upper and lower case letters
Intrinsic function CHAR and ICHAR convert between character strings and ascii codes and visa versa.
*Reading with a * format will allow three strings to be read on the same line i.e.,*
read(,*) string1, string2, string3*
will allow all names to be on the one line.
Writing with a format of a single a (instead of a10 for example) will output only the number of characters in the string to be output. To avoid extra spaces, only print the number of characters needed using the 1:N feature where N is the number of characters needed.

Again a nominally easy program but we will clearly need a few utility subroutines to perform various tasks. From the question, the main ones will be:

(1) A subroutine that will convert a string to upper case. Examination of the ASCII codes shows that the only difference between upper and lower case is that the

upper case symbol for a letter is 32 less than the lower case value. (man ascii on most systems will print the ASCII table). Therefore by use of ICHAR to get the ASCII numeric value and CHAR to convert the ASCII numeric value back to a character, we can convert from lower to upper case. But we do need to be careful: Only the characters from a-z should be converted.

(2) Since we need to do other manipulations of strings, we will probably need something that finds the length of the used portion of a string, i.e., even though a string may be declared to a certain length only some of the string is filled with non-blank characters (in class we discussed that fortran pads strings with blanks if a shorter length string is assigned to a longer string

(3) Since the question calls for re-arranging the names, a routine that extracts "words" from a string will be useful. An alternative would be to read the parts of the name into different strings in which case, this routine would not be needed. This second approach is used in the homework solution.

For items 2 and 3 above, the routines will work by finding the positions in strings of blank and non-blank characters and using the fortran string(n:m) feature to extract those parts of the string.

Code design

- Ask the user for the string to be converted and read the user input.
- Check for IOSTAT errors reading the string
- Check that the string given is not of zero-length (lenline function).
- Convert to upper case (tocaps subroutine--- this could be implemented as a function since there is only one return)
- Extract each part from it, namely first name, middle initial and last name. (GetName routine)
- Check the lengths of the parts of the names to see if zero length.
- If the last name is zero length, then this implies that only two parts of the name were entered so take the second part of name to be last name and make the middle initial a question mark since it does not seem to have been entered.
- If the last name still seems to be zero length, then assume only one name was entered and so take this to the last name and use question marks for the first name and middle initials.
- Convert to upper case the first letter of each part of the names (in case no upper case given by user).
- Compute the lengths of each name part and compute the size of banner needed
- Output banner and names using the first character only of the middle name.

Routine tocaps(in_string, out_string)
Get the minimum length of string passed (so that we don't exceed bounds)
Loop over each character in string
    If the input character is between 'a' and 'z'
Subtract 32 from ASCII code and convert back to character

Endif
End loop
Function lenline(inline)
Returns the length of the used portion (non-blank character) of a string
Get the full length of the string
Start at the end of the string working backwards to find a non-blank character. Make sure we do not go past the start of the string
Return the number of characters to last non-blank part of the string

Fortran90/gfortran have a len_trim function which returns the length of the non-blank portion of a string and a function trim which returns the non-blank part of a string (non-blank is the trailing blanks in the string)

Routine getname(instring, position, nextword)
Extracts the next full word from string starting from position in string. In normal use the position would start at 1 or less.
Check the position passed into the routine. If it is less than zero (actually a bad call) set it to zero. If it is greater than or equal to the full length of the string then we are at the end of the string already, so keep the position at the end of the string and return a blank string.
Start working up the string looking for the next non-blank character. This marks the start of the next word. Once this has been found, keep moving up the string to find the next blank, which will mark the end of the word. In both loops make sure that we don't go past the end of the string. Copy the part of the character string corresponding the next word unless the bounds are such that there is no next word in which case the word is returned as blank.
The fortran code I used is linked to http://geoweb.mit.edu/~tah/12.010/HW02_02_11.f

```
TAHComputer[290] gfortran HW02_02_11.f -o HW02_02
TAHComputer[291] HW02_02
Enter your names (First, middle, last) James quincy adams
******************
* Adams, James Q. *
******************
Second method: You will need to enter all 3 names
Enter first, middle and last name james quincy adams
++++++++++++++++++
+ Adams, James Q. +
++++++++++++++++++
```

Question (3): *(50-points) Write a Fortran program that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as*

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where H(x) is the height of the path above the starting height, S is a slope in m/m, A and B are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is l. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your program can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The program can assume a constant power scenario and constant force at low velocities.

As a test of your program use the following constants to compute:

- *(a) Time to travel and energy used to travel 10 km along a path specified by S=0.01, A=5.0 m, B=0.0 m and l= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.*
- *(b) The position and velocity of the bike tabulated at a 100-second interval.*

Assume the following values

C_d = 0.9
C_r = 0.007
A_r = 0.67 m^2
ρ = 1.226 kg/m^3
g = 9.8 m/s^2
M_r = 80 kg

Your answer to this question should include:

- *(a) The algorithms used and the design of your program*
- *(b) The Fortran program source code (I will compile and run your programs).*
- *(c) The results from the test case above.*

With the equations given above, we can compute the forces acting on the bike (gravity, drag, rolling friction and the rider input force) and using the mass of the bike and rider, compute accelerations. By integrating the accelerations we generate velocity changes

and by integrating the velocity changes we get position changes. Since the forces depend on velocity, we need to integrate in small steps. Precisely how small we discuss below. This problem is basically an integration problem. It can also be posed as the solution to a second-order differential equation and in later homework (and languages) we will how to solve it this way.

There are two basic ways of approaching this problem:
(1) Assume that the bike remains attached to the track (roller coaster problem) and resolve the forces into the direction of the track at the location of the bike and assume that the force perpendicular to this are balanced and can be neglected. In this solution, the integration is along the track and because the track description is given as a function of horizontal distance and that the total distance traveled is the horizontal distance (and not the distance along the track), the integration will need to keep track of both the distance along the track and the horizontal distance. For a small change in distance, the horizontal distance increment is dS cos(theta) where theta is the local slope.
(2) Compute the complete force balance on the bike and solve the equations in two dimensions. In this mode of solution, the bike does not need to stay on the ground. In this solution, we need to add gravity acting vertically down, the normal force from the track surface which largely compensates gravity but because of local slopes will also have a component that acts in the direction of motion of the bike, and finally because of the curvature of the track, the centrifugal and centripetal forces need to be considered. If the force balance is computed correctly and the bike is slow enough that is does not leave the track, the integration of acceleration and velocity to yield position should trace out the shape of the track.
Both of the above solutions are given is the solution. In both cases, we also add code that allows the accuracy of the integrations to be specified.

Since in both solutions above we need integration we should go to the library and read about numerical integration. Notice that there are two parts to integration problems: (1) you need to do the integration and (2) the program has to determine the accuracy of the integration. In particular, if the user requests low accuracy integration, then the program should run quicker. In reading about numerical integration both methods for integration and the way of judging accuracy must be determined. You need to know the techniques well enough to implement in a program.
As discussed in the solution to Homework 1, the basic equation being integrated here the acceleration of the body. The tricky part of the problem is that the acceleration depends on the velocity of the body. In the full approach, (2) above, we are solving a pair of non-linear ordinary differential equations in this problem. We have:
$\ddot{x} - k(\dot{x})^2 - c(\dot{x}) - f(g) = 0$
where x is a vector of the x and z coordinates of the object, and the derivatives are with respect to time. The k term here is drag, c($\dot{x}$) is the centripetal term and f(g) is the gravity and surface normal forces. In general, the integral above can not be solved analytically because the functions c and f depend on the track shape.

I consulted Numerical Recipes (for the discussion of numerical integration, not the programs) and Abramowitch and Stegum. The latter book gives the Runge-Kutta formula for numerically integrating an equation of the form y''(x) = f(x,y,y') where in our case x is time, y is position, y' is velocity and y'' is acceleration.

$$y_{n+1} = y_n + h[y'_n + \frac{1}{6}(k_1 + k_2 + k_3)] + O(h^5)$$

$$y'_{n+1} = y'_n + \frac{1}{6}[k_1 + 2k_2 + 2k_3 + k_4]$$

$$k_1 = hf(x_n, y_n, y'_n)$$

$$k_2 = hf(x_n + h/2, y_n + hy'_n/2 + hk_1/8, y'_n + k_1/2)$$

$$k_3 = hf(x_n + h/2, y_n + hy'_n/2 + hk_1/8, y'_n + k_2/2)$$

$$k_4 = hf(x_n + h, y_n + hy'_n + hk_3/2, y'_n + k_3)$$

The equations above give the position (y) and velocity (y') at time step n+1 in the numerical integration based on the values at time step n. The length of the time step is h. The O(h^5) statement means that the error in the integration decreases as step size to the 5th power. The simple approach to the numerical integration in Euler's method in which the acceleration (function f) is computed for the position and velocity at the start of the step and is assumed constant for the distance traveled and velocity change during the step. The above equation is an approach (one of many different ones available) in which the constant acceleration assumption is replaced with accelerations computed at different points in the time step. A very simple application of this approach is consider where gravity will be computed. Since the velocity of the body is known at the start of the step, an approach height at the end of the step can be computed (assuming constant velocity) and constant value of gravity used could be computed at the mid-point of the height at the beginning of the step and the approximate position at the end of the step. Similarly, rather than computing drag using the initial velocity only, the change in velocity can be estimated and the drag computed using the average velocity of the drag during the step. The Runge-Kutta equations are derived using these ideas. The Runge-Kutta integration method allows the trajectory to be computed based in an initial velocity. Our problem is two-dimensional and in f90 we could implement the equations above uses two element arrays for y and y'. In f77, we could compute each component separately, or we could use a technique commonly used in 2D problems. We can make y and y' complex data types where the real component is the x-coordinate and the imaginary component is the z coordinate.

One complication in the integration here is that we are not integrating for a finite length of time but rather until the x-coordinate reaches a set distance. This is a bit tricky for the last integration step where the time interval until the objects is not likely to be the time step being used for the integration. The implementation in the solution is to integrate until the bike goes just past the end of the distance. Smaller integration step sizes are made as we approach the end. Based on how far back in time in that last step,

the last step of the integration is done with a smaller step size. The last step is repeated with smaller step sizes until the bike is within the tolerance of the distance for the accuracy of the calculation. The energy integration is done with a a 3-point Newton-Cotes Methods (also called Simpson's rule).

The one-dimensional solution used the same approach except that the horizontal coordinate needs also the integrated. The integration is built into the Runge-Kutta integration.

The basic flow of the program is to:

-Read the inputs fro the user. These reads are ordered such that those quantities most likely to be changed by the user. These values have defaults and so the user only needs to use /<cr> to adopt the defaults. – Initialize the variables needed in the program

- Trial integrations with decreasing step sizes to determine the step size needed to hit the ground within a specific accuracy. In the standard example we try for 10 mm precision in distance accuracy.
  The integration routine is called Int2d for the two-dimensional integration and Int1d for the one-dimensional version. The routine called accel returns the acceleration at a specific time, position and velocity (position is needed to compute slopes and surface curvature in Int2d (the latter is needed for centripetal force calculation). Notice in the 2D version we treat the X and Z coordinates, velocities and accelerations as complex numbers. This allows the acceleration function to return a complex result that contains the X and Z acceleration. In the Runge-Kutta integration equations, I just need one set of expressions that do the complex calculations. (Here I just need to be careful to only multiply by real values and to add and subtract complex values.)
- Once the step size is set, run with the final step size and output results. (This later step is a little inefficient since it repeats an earlier calculation but with output generated this time. The output of the trajectory lines start with T[*] and I can use this to extract using grep the trajectory of the bike.

Out put the final velocity and the trajectory (optional for making plots)

The program in implemented is given in
http://geoweb.mit.edu/~tah/12.010/HW02_03-2D_11.f
for the full two-dimensional calculation and in
http://geoweb.mit.edu/~tah/12.010/HW02_03-1D_11.f
for the alternative formulation that uses the one-dimensional integration. Both programs also uses an include file
http://geoweb.mit.edu/~tah/12.010/FBike.h
that contains parameters for gravity values, air density etc and a common block that contains the “fixed:” variables in the program (object mass, drag coefficient etc.)
We also include a

http://geoweb.mit.edu/~tah/12.010/HW02_03_11.in file that contains input for the program. (Using / to respond to the questions asked by the program will also generate the standard example output.)

Example output for test case:

```
(HW02_03-2D < HW02_03_11.in >! HW02_03_11.out can also be used.  The run bellow
uses just default values.  These are the values stored in HW02_03_11.in)


% gfortran HW02_03-2D_11.f -o HW02_03-2D
% HW02_03-2D

12.010 Program to solve Bike Motion problem
Given track characteristics and rider/bike properties the time, energy and path
are computed

Program Parameter Input. [Defaults are printed use /<cr> to accept default
Length of track (km) and error (mm) [10.000 km  10.0 mm] /
Track Slope, Sin and Cos amplitudes (m) and wavelenghth (km) [Defaults 0.001
5.00 m  0.00 m    2.00 km] /
Rider/Bike Mass (kg), Area (m**2), Drag and rolling Coefficient ,[ 80.00 kg,
0.670 m^2,   0.90 and 0.0070] /
Rider Power (W) and max force (N) [100.00 Watts, 20.00 N] /
Output interval, zero for none (default   100.00 s) /

++++++++++++++++++
PROGRAM PARAMETERS
++++++++++++++++++
Length of track  10.000 (km) and error   10.0 (mm)
Track Slope 0.001 Sin and Cos amplitudes  5.00  0.00 (m) and wavelenghth   2.00
(km)
Rider/Bike Mass  80.00(kg), Area  0.670 (m**2),Drag and rolling Coefficient
0.90 0.0070
Rider Power 100.00 (Watts) and max force  20.00 (N)
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Step size    0.100E+01 s, Times  2255.1473 2255.1669 s  Error     195.74 mm
Step size    0.500E+00 s, Times  2255.1669 2255.1728 s  Error      58.73 mm
Step size    0.250E+00 s, Times  2255.1728 2255.1739 s  Error      11.37 mm
Step size    0.125E+00 s, Times  2255.1739 2255.1741 s  Error       2.50 mm
O*    Time          X_pos          Z_pos          X_vel          Z_vel     Energy
O*    (sec)          (m)            (m)           (m/s)          (m/s)    (Joules)
O      0.000         0.0000         0.0000         0.0000         0.0000         0.00
O    100.000        79.9901         1.3233         1.4887         0.0241      1600.02
O    200.000       306.7628         4.4134         3.3670         0.0335      6135.90
O    300.000       826.7974         3.4153         6.7976        -0.0846     15409.05
O    400.000      1513.8127        -3.4814         5.8770         0.0099     25409.05
O    500.000      1951.6809         1.1956         2.9595         0.0489     33914.45
O    600.000      2202.4203         5.1719         2.6542         0.0362     38929.87
O    700.000      2608.5166         7.3207         5.8297        -0.0248     46841.18
O    800.000      3292.8975        -0.6856         6.8882        -0.0586     56841.18
O    900.000      3834.8273         1.3551         3.8825         0.0568     66456.49
O   1000.000      4117.3686         5.9193         2.3535         0.0368     72108.05
O   1100.000      4428.1180         9.3010         4.5333         0.0205     78323.45
O   1200.000      5046.5582         4.3178         7.2497        -0.1054     88273.31
O   1300.000      5681.6964         1.4743         4.9737         0.0472     98273.04
O   1400.000      6035.2522         6.5876         2.4808         0.0412    105344.91
O   1500.000      6294.1703        10.2844         3.3205         0.0347    110523.81
O   1600.000      6803.7014         9.6952         6.7144        -0.0793    119705.43
O   1700.000      7493.5363         2.4943         5.9865         0.0041    129705.43
O   1800.000      7941.4394         7.0264         3.0334         0.0499    138347.80
O   1900.000      8193.4336        11.0479         2.6045         0.0362    143388.33
O   2000.000      8588.7675        13.3952         5.7249        -0.0190    151137.56
O   2100.000      9269.2214         5.5265         6.9632        -0.0656    161137.56
O   2200.000      9821.3690         7.1600         3.9921         0.0571    170825.51
```

```
O   2255.174     10000.0004         9.9995         2.6501         0.0443    174398.59

Time to travel 10.000 km,  2255.17 seconds,   0.63 hrs
Rider Energy    174398.59 Joules,    41654.387 Calories
Potential         7839.61 Kinetic       281.00 Joules

Final Velocity  2.650 0.044 m/sec
```

Results for Test case:

Program	Time	Energy used
2-D	2255.17 seconds	174407.94 Joules
1-D	2255.06 seconds	174407.62 Joules

Plot below shows results from program. To get the time series of the values use a 1-second output can grep on lines starting with '^O' as shown below:

```
% HW02_03-2D < HW02_03_11.in | grep '^O | awk '{print substr($0,3)}' >! 2d.dat
```

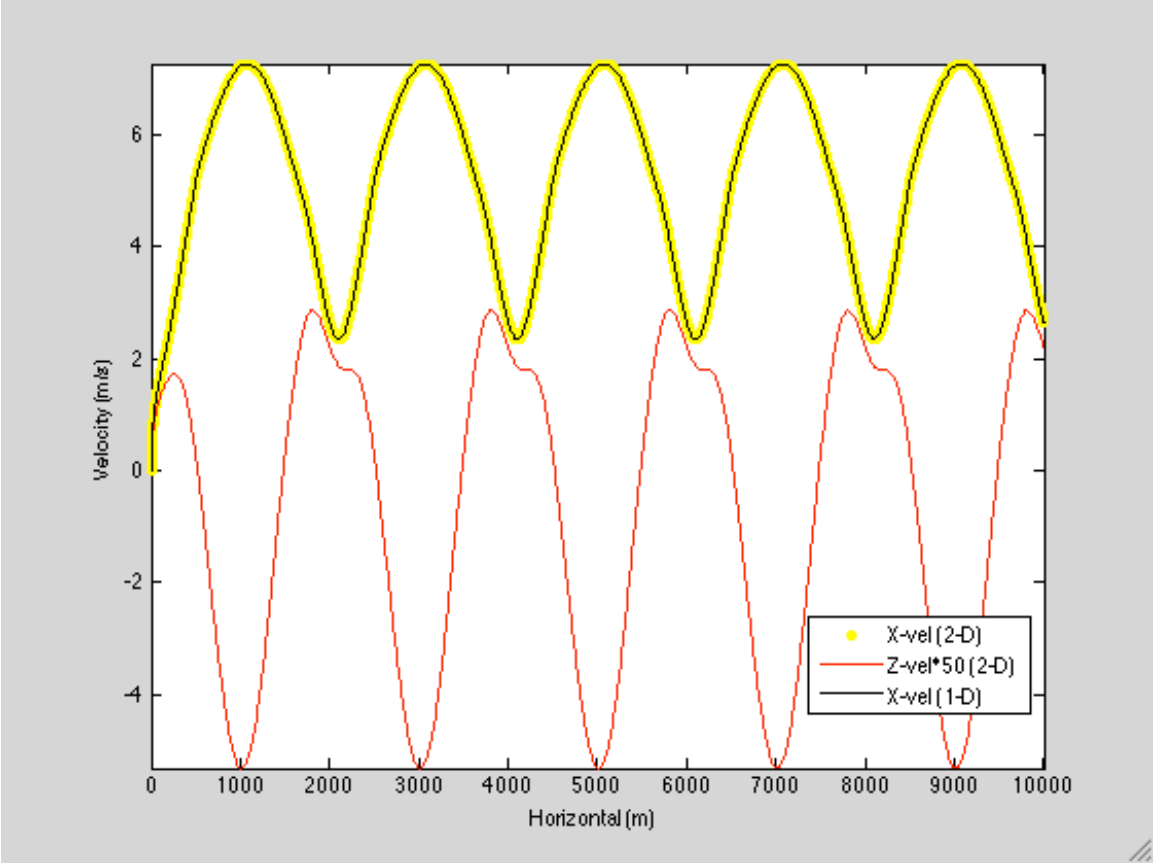


Figure 1: X and Z velocities of bike as a function of time. The X velocity for the 2-D solution the shown with yellow dots and from the 1-D solution with a black line. Notice that Z velocity is multiplied by 50 because this velocity is much smaller than the X velocity.

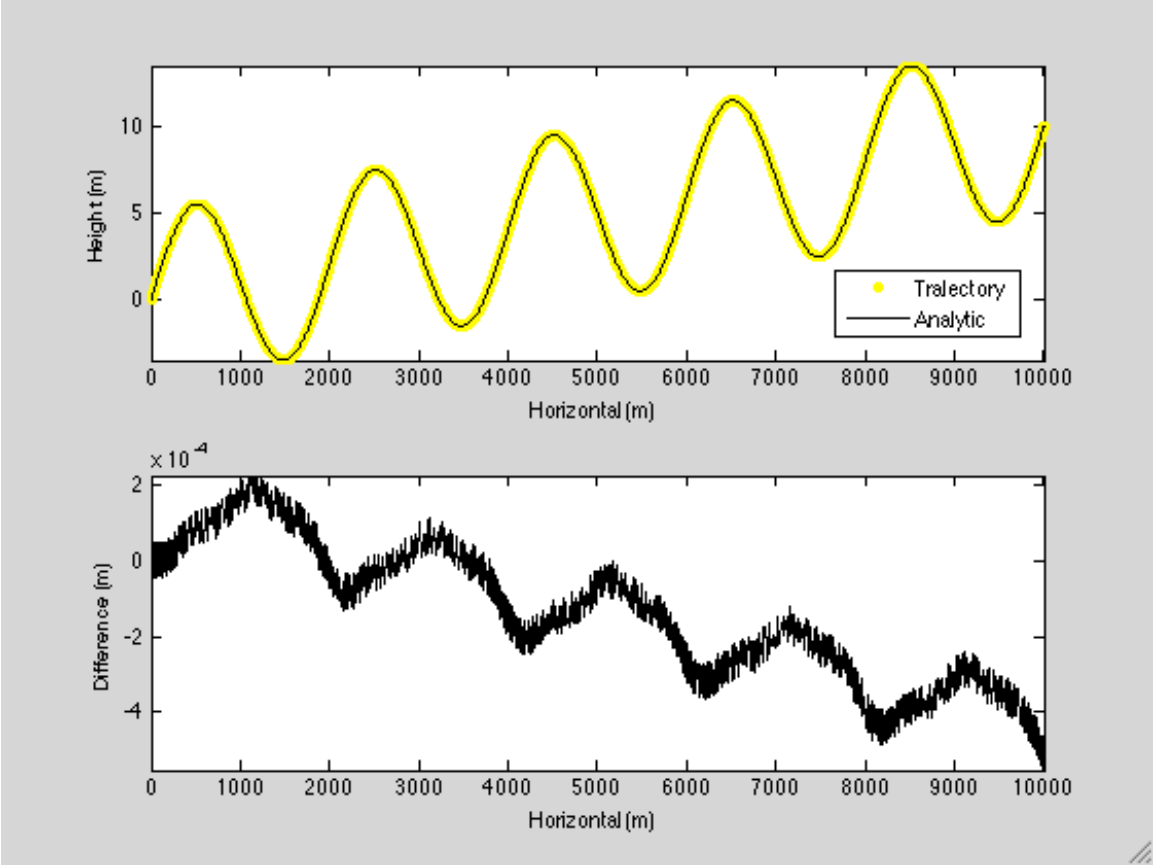


Figure 2: Top: Path followed by bike computed by integrating accelerations (Height (m)), yellow dots, and analytic solution (black). Bottom: the difference between the theoretical path and this path shown on the right hand scale. The differences are <1 mm. and arise from accumulated rounding error.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #3 **Due Thursday, November 03, 2011**

Question (1): (25-points) (a) Write, compile and run a C/C++ program which generates a table of error function (erf) and its derivatives for real arguments (*z*) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the program and give an example of the output.
(b) How would you change this program if 10 significant digits were required?
C/C++ source code should also be supplied

Question (2): (25-points).
Write a program that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The program should not be specific to the lengths of your name (ie., the program should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.
Hints:
You might look for toupper and tolower as part of the ctype.h header file.

Question (3): (50-points) Write a C/C++ program that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where *H(x)* is the height of the path above the starting height, *S* is a slope in m/m, *A* and *B* are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your program can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The program can assume a constant power scenario and constant force at low velocities.

As a test of your program use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.

(b) The position and velocity of the bike tabulated at a 100-second interval.

Assume the following values

C_d = 0.9

C_r = 0.007

A_r = 0.67 m^2

r = 1.226 km/m^3

g = 9.8 m/s^2

M_r = 80 kg

Your answer to this question should include:

(a) The algorithms used and the design of your program

(b) The C/C++ program source code (I will compile and run your programs).

(c) The results from the test case above.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #3 **Due Thursday, November 03, 2011**

Question (1): *(25-points) (a) Write, compile and run a C/C++ program which generates a table of error function (erf) and its derivatives for real arguments (z) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).*

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the program and give an example of the output.
(b) How would you change this program if 10 significant digits were required?
C/C++ source code should also be supplied

Solution:

The algorithm used here is the same as that used in the Fortran solution. However we have added here the ability to pass the desired accuracy of the calculation through the runstring. The default accuracy is 1e-6. Since the erf function is part of the math library (-lm option need when compiled and linked), this version of the program outputs the difference between the series expansion and the math library erf function.

The source code is http://geoweb.mit.edu/~tah/12.010/HW03_01_11.c

Example output:

```
TAHComputer[167] gcc HW03_01_11.c -o HW03_01_11
TAHComputer[168] HW03_01_11
------------------------------------------------------------
TABLE OF ERROR FUNCTION (ERF) within 1.00e-06 AND FIRST DERIVATIVE
------------------------------------------------------------
| Argument |     ERF    | d(ERF)/dx |     ERFC     |    Error    |
|__________|____________|___________|______________|_____________|
|  -3.000  |  -0.99998  |   0.00014 |   -0.99998   |  -1.43e-07  |
|  -2.750  |  -0.99990  |   0.00059 |   -0.99990   |  -1.45e-07  |
|  -2.500  |  -0.99959  |   0.00218 |   -0.99959   |   5.31e-08  |
|  -2.250  |  -0.99854  |   0.00714 |   -0.99854   |   1.30e-07  |
|  -2.000  |  -0.99532  |   0.02067 |   -0.99532   |  -1.10e-07  |
|  -1.750  |  -0.98667  |   0.05277 |   -0.98667   |   1.46e-07  |
|  -1.500  |  -0.96611  |   0.11893 |   -0.96611   |   5.00e-08  |
|  -1.250  |  -0.92290  |   0.23652 |   -0.92290   |   2.24e-08  |
|  -1.000  |  -0.84270  |   0.41511 |   -0.84270   |   1.37e-08  |
|  -0.750  |  -0.71116  |   0.64293 |   -0.71116   |   1.17e-08  |
|  -0.500  |  -0.52050  |   0.87878 |   -0.52050   |   1.43e-08  |
|  -0.250  |  -0.27633  |   1.06001 |   -0.27633   |  -2.02e-10  |
|   0.000  |   0.00000  |   1.12838 |    0.00000   |   0.00e+00  |
|   0.250  |   0.27633  |   1.06001 |    0.27633   |   2.02e-10  |
|   0.500  |   0.52050  |   0.87878 |    0.52050   |  -1.43e-08  |
|   0.750  |   0.71116  |   0.64293 |    0.71116   |  -1.17e-08  |
```

```
|   1.000    |   0.84270  |    0.41511   |    0.84270    | -1.37e-08  |
|   1.250    |   0.92290  |    0.23652   |    0.92290    | -2.24e-08  |
|   1.500    |   0.96611  |    0.11893   |    0.96611    | -5.00e-08  |
|   1.750    |   0.98667  |    0.05277   |    0.98667    | -1.46e-07  |
|   2.000    |   0.99532  |    0.02067   |    0.99532    |  1.10e-07  |
|   2.250    |   0.99854  |    0.00714   |    0.99854    | -1.30e-07  |
|   2.500    |   0.99959  |    0.00218   |    0.99959    | -5.31e-08  |
|   2.750    |   0.99990  |    0.00059   |    0.99990    |  1.45e-07  |
|   3.000    |   0.99998  |    0.00014   |    0.99998    |  1.43e-07  |
|____________|____________|______________|_______________|____________|
```

As in the Fortran code, this code takes an accuracy value so the number of digits to be output would need to be changed.

Question (2): *(25-points).*
Write a program that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The program should not be specific to the lengths of your name (ie., the program should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.
Hints:
You might look for toupper and tolower as part of the ctype.h header file.

Solution:
This solution is similar to the Fortran code solution. Care should be taken in using scanf for input. In this code fscanf is used so that the maxium number of bytes to be read can be specified. The code also checks the number of characters read but by using fscanf the number should never exceed the maximum allowed.

The solution code is http://geoweb.mit.edu/~tah/12.010/HW03_02_11.c

Example run.

```
TAHComputer[170] gcc HW03_02_11.c -o  HW03_02_11
TAHComputer[171]  HW03_02_11

Enter your names (First, middle, last) tHoMaS aBrAm hErrIng
**********************
* Herring, Thomas A. *
**********************
```

Question (3): *(50-points) Write a C/C++ program that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as*

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where H(x) is the height of the path above the starting height, S is a slope in m/m, A and B are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your program can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The program can assume a constant power scenario and constant force at low velocities.

As a test of your program use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.

(b) The position and velocity of the bike tabulated at a 100-second interval.

Assume the following values

C_d = 0.9

C_r = 0.007

A_r = 0.67 m^2

ρ = 1.226 kg/m^3

g = 9.8 m/s^2

M_r = 80 kg

Your answer to this question should include:

(a) The algorithms used and the design of your program

(b) The C/C++ program source code (I will compile and run your programs).
(c) The results from the test case above.

Solution:

This solution is very close to the Fortran HW03_03_11-1D solution (i.e., the bike is treated like a roller coaster and the problem is solved in 1-D while accounting for the difference between the sloped and horizontal distances.

As in the Fortran code a header file is used (CBike.h) and this contains definitions and global variables that are accessible in all functions. Prototype function lines are included at the top of the source code.

Notice some of the changes that needed to be made to the code. In particular, we needed to change the function call to nint in the Fortran code to ceil in the C code, and we needed to change the abs call to fabsl. Specifically here if we left the abs call as coded, the wrong answers were generated because abs is the integer values only. We also changed the input so that the user can simply say no and keep the default values. We also coded the input so that the same use of / as in Fortran code was used.

The solution is http://geoweb.mit.edu/~tah/12.010/HW03_03_1D_11.c and http://geoweb.mit.edu/~tah/12.010/CBike.h

Example with defaults.

```
TAHComputer[204] gcc HW03_03_1D_11.c -o  HW03_03_1D_11
TAHComputer[205] HW03_03_1D_11

12.010 Program to solve Bike Motion problem
Given track characteristics and rider/bike properties
the time, energy and path are computed.

Do you want to change defaults (y/n) n

PROGRAM PARAMETERS
++++++++++++++++++
Length of track  10.000 km and error   10.0 mm
Track Slope 0.001 Sin and Cos amplitudes  5.00  0.00 (m) and wavelenghth
2.00 (km)
Rider/Bike Mass  80.00 (kg), Area  0.670 (m**2), Drag and rolling
Coefficient   0.90 0.0070
Rider Power 100.00 (Watts) and max force  20.00 (N)
Output Interval 100.00 (s)

++++++++++++++++++
Step 1.000000 Times 2255.063413 2255.063412 Delta (ms) 0.001372
O*    Time          X_pos        S_pos               S_vel     Energy
O*    (sec)          (m)          (m)                (m/s)     (Joules)
O      0.000         0.0000         0.0000         0.0000         0.00
O    100.000        80.0297        80.0407         1.4896      1600.80
```

O	200.000	306.9129	306.9453	3.3689	6138.93
O	300.000	827.1168	827.1622	6.7990	15413.85
O	400.000	1514.1006	1514.1889	5.8755	25413.85
O	500.000	1951.8144	1951.9321	2.9589	33917.06
O	600.000	2202.5693	2202.7186	2.6557	38932.85
O	700.000	2608.8907	2609.0514	5.8317	46847.61
O	800.000	3293.3369	3293.5471	6.8874	56847.61
O	900.000	3835.0718	3835.2993	3.8807	66461.32
O	1000.000	4117.5262	4117.7907	2.3544	72111.21
O	1100.000	4428.4800	4428.7645	4.5370	78330.75
O	1200.000	5047.1325	5047.4485	7.2508	88281.45
O	1300.000	5682.1018	5682.4419	4.9717	98281.13
O	1400.000	6035.4481	6035.8262	2.4806	105348.84
O	1500.000	6294.4797	6294.8846	3.3237	110530.07
O	1600.000	6804.3476	6804.7643	6.7171	119715.19
O	1700.000	7494.1230	7494.5844	5.9834	129715.19
O	1800.000	7941.7253	7942.2147	3.0317	138353.52
O	1900.000	8193.7065	8194.2280	2.6066	143393.84
O	2000.000	8589.4172	8589.9508	5.7284	151148.99
O	2100.000	9270.0071	9270.5896	6.9615	161148.99
O	2200.000	9821.8172	9822.4164	3.9886	170834.47
O	2255.063	10000.0004	10000.6222	2.6506	174490.72

Time to travel 10.000 km, 2255.06 seconds, 0.63 hrs
Rider Energy 174490.72 Joules, 41676.393 Calories
Kinetic 281.02 Joules
Final Velocity 2.651 m/sec

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #4 **Due Thursday, November 17, 2011**

All solutions can be included in a single NoteBook with the results in different sections or sub-sections.

Question (1): (25-points) (a) Write Mathematica NoteBook which generates a table of error function (erf) and its derivatives for real arguments (*z*) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the NoteBook and give an example of the output.
(b) How would you change this NoteBook if 10 significant digits were required? Mathematica NoteBook should also be supplied

Question (2): (25-points).
Write a NoteBook that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The NoteBook should not be specific to the lengths of your name (ie., the NoteBook should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.

Question (3): (50-points) Write a Mathematica NoteBook that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where *H(x)* is the height of the path above the starting height, *S* is a slope in m/m, *A* and *B* are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your NoteBook can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The NoteBook can assume a constant power scenario and constant force at low velocities.

As a test of your NoteBook use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.01, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.
(b) The position and velocity of the bike tabulated at a 100-second interval.
(c) Add graphics to your NoteBook which plots the velocity of the bike as a function of time and position along the path.

Assume the following values

C_d = 0.9

C_r = 0.007

A_r = 0.67 m^2

r = 1.226 km/m^3

g = 9.8 m/s^2

M_r = 80 kg

In this case, the Mathematica NoteBook will not be of the type used for fortran and C/C++. Look at the documentation on NDSolve for this problem.
Your answer to this question should include:

(a) The algorithms used and the design of your NoteBook
(b) The Mathematica NoteBook with your code and solution (I run your NoteBook).
(c) The results from the test case above.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 HW 4 2011

This latest version of Mathematica has some issues with copying and pasting equations and graphics from Microsoft Word. It also seems not to be able to reproduce subscripts and superscripts from word, which previously was able to do. It is not clear if the incompatibility is from the Microsoft Word or the Mathematica side. This is another indication of how changes in versions of software can have unintended consequences.

(Question 1): (25-points) (a) Write Mathematica NoteBook which generates a table of error function (erf) and its derivatives for real arguments (z) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).

$$\text{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)

The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the NoteBook and give an example of the output.

(b) How would you change this NoteBook if 10 significant digits were required?

Mathematica NoteBook should also be supplied

- Solution: Given that the error function is defined in *Mathematica* themost tricky part of the this problem is formarting the ourput. There are many ways to approach the output problem.

In[1]:=

```
Off[General::spell1];
(* Stops message about head looking like a command*)
head = {"Argument", "Erf[z]", "dErf[z]/dz"};
derf[arg_] := 2 / Sqrt[Pi] * Exp[-x^2];
ents =
  Table[{N[SetAccuracy[x, 10], 3],
    N[SetAccuracy[Erf[x], 10], 6],
    N[SetAccuracy[derf[x], 6], 6]}, {x, -3, 3, 0.25}];
full = Insert[ents, head, 1];
TableForm[full, TableAlignments → Right]
```

Out[6]//TableForm=

Argument	Erf[z]	dErf[z]/dz
-3.00	-0.999978	0.00014
-2.75	-0.999899	0.00059
-2.50	-0.999593	0.00218
-2.25	-0.998537	0.00714
-2.00	-0.995322	0.02067
-1.75	-0.986672	0.05277
-1.50	-0.966105	0.118930
-1.25	-0.922900	0.23652
-1.00	-0.842701	0.41511
-0.750	-0.711156	0.64293
-0.500	-0.520500	0.87878
-0.250	-0.276326	1.06001
$0.\times10^{-10}$	$0.\times10^{-10}$	1.12838
0.250	0.276326	1.06001
0.500	0.520500	0.87878
0.750	0.711156	0.64293
1.00	0.842701	0.41511
1.25	0.922900	0.23652
1.50	0.966105	0.118930
1.75	0.986672	0.05277
2.00	0.995322	0.02067
2.25	0.998537	0.00714
2.50	0.999593	0.00218
2.75	0.999899	0.00059
3.00	0.999978	0.00014

- (b) Output of table to 10 deciminal places.

In[22]:=

```
Off[General::spell1]; (* Stops message about head looking like a command*)
head = {"Argument", "Erf[z]", "dErf[z]/dz"};
derf[arg_] := 2 / Sqrt[Pi] * Exp[-x^2];
ents = Table[{N[SetAccuracy[x, 10], 3], N[SetAccuracy[Erf[x], 10], 10],
        N[SetAccuracy[derf[x], 10], 10]}, {x, -3, 3, 0.25}];
full = Insert[ents, head, 1];
TableForm[full, TableAlignments → Right]
```

Out[26]//TableForm=

Argument	Erf[z]	dErf[z]/dz
-3.00	-0.999977910	0.000139253
-2.75	-0.999899378	0.0005862772
-2.50	-0.999593048	0.002178284
-2.25	-0.998537283	0.0071423190
-2.00	-0.995322265	0.0206669854
-1.75	-0.986671671	0.0527749959
-1.50	-0.9661051465	0.1189302892
-1.25	-0.9229001283	0.2365211224
-1.00	-0.8427007929	0.415107497
-0.750	-0.7111556337	0.642931069
-0.500	-0.5204998778	0.878782579
-0.250	-0.2763263902	1.060014129
$0.\times10^{-10}$	$0.\times10^{-10}$	1.128379167
0.250	0.2763263902	1.060014129
0.500	0.5204998778	0.878782579
0.750	0.7111556337	0.642931069
1.00	0.8427007929	0.415107497
1.25	0.9229001283	0.2365211224
1.50	0.9661051465	0.1189302892
1.75	0.986671671	0.0527749959
2.00	0.995322265	0.0206669854
2.25	0.998537283	0.0071423190
2.50	0.999593048	0.002178284
2.75	0.999899378	0.0005862772
3.00	0.999977910	0.000139253

Question (2): (25-points). Write a NoteBook that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a

period after the middle initial. If the names start with lower case letters, then these should be capitalized. The NoteBook should not be specific to the lengths of your name (ie., the NoteBook should work with anyone's name.

As an example. An input of

thomas abram herring

would generate:

Herring, Thomas A.

- This problem is not too bad to solve. This solution works in >5.0 *Mathematica* and does explicitly some things such as splitting a string apart that are now *Mathematica* commands (StringSplit).

In[27]:=

```
(* Define a function that will convert chararacter of a string to upper case *)
confirst[a_] := StringReplacePart[a, ToUpperCase[StringTake[a, {1}]], {1, 1}];
(* Get the name from the user*)
inname = InputString["Enter Name (first, middle, last) "];
(* Convert whole string to lower case*)
fullname = ToLowerCase[inname];
(* Now get the list of blanks in the string*)
posblanks = StringPosition[fullname, " "];
(* Get the position of first blank*)
posfirst = Extract[Extract[posblanks, 1], 1];
firstname = StringTake[fullname, posfirst - 1];
firstname = confirst[firstname]; (* Use our confirst routine *)
(* Get Middle Name *)
posmid = Extract[Extract[posblanks, 2], 1];
midinit = StringTake[fullname, {posfirst + 1, posfirst + 1}];
midinit = confirst[midinit];
(* Get Last Name *)
lastname = StringTake[fullname, {posmid + 1, StringLength[fullname]}];
lastname = confirst[lastname];
(* Output the string *)
finalname = lastname <> ", " <> firstname <> " " <> midinit <> ".";
outline = "Input name " <> inname <> " converted to: " <> finalname;
(* Output the results*)
Print[outline];
```

```
Input name thomas abram herring converted to: Herring, Thomas A.
```

Write a Mathematica NoteBook that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where *H(x)* is the height of the path above the starting height, *S* is a slope in m/m, *A* and *B* are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where *Ar* is the cross-sectional area of the rider, *Cd* is the drag coefficient, r is the density of air and *V* is the velocity of the bike. For the rolling drag, *Mr* is the mass of the rider and bike, *g* is gravitation acceleration and *Cr* is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where *Fr* is the force produced by the rider, *Pr* is power used by the rider and *V* is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your NoteBook can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The NoteBook can assume a constant power scenario and constant force at low velocities.

As a test of your NoteBook use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of *Pr* =100Watts

and a maximum force available of 20N.

(b) The position and velocity of the bike tabulated at a 100-second interval.

(c) Add graphics to your NoteBook which plots the velocity of the bike as a function of time and position along the path.

Assume the following values

Cd = 0.9

Cr = 0.007

Ar = 0.67 m2

ρ = 1.226 kg/m3

g = 9.8 m/s2

Mr = 80 kg

In this case, the Mathematica NoteBook will not be of the type used for fortran and C/C++. Look at the documentation on NDSolve for this problem.

Your answer to this question should include:

(a) The algorithms used and the design of your NoteBook

(b) The Mathematica NoteBook with your code and solution (I run your NoteBook).

(c) The results from the test case above.

- There are several ways of approaching this problem and two solution are presented below. The problem itself is dvided into a number of cells that allow parts of problem to be re-executed. There two basic steps to the solution:
(1) Use NDSolve to solve the second order differential equation that describes the problem. The solution effectively becomes an equation that returns values at any specified time.

(2) Use FindRoot to determine when the bike has reached the end of track.

- Set up the defaults first. This cell should be evaluated

```
Clear[t, x, xd, hx, vx, xp, xd, th, dacc];
cd = 0.90 ;
(*Print["Parameter cd ",cd]*)
cr = 0.007;
area = 0.67; (* m^2 *)
mass = 80; (* kg *)
prider = 100; (* Power Watts*)
fmax = 20;
slope = 0.001;
as = 5.0;
bs = 0.0;
lambda = 2000;
tracklen = 10 000;
outint = 100.0;

(* Define the acceleration functions we will need*)
grav = 9.8 ;
rhoair = 1.226;
Print["Default Values set"]
```

```
Default Values set
```

- Now allow the user to enter values of there own. This cell does not need to be evaluated if the default values are desired.

In[82]:=
```
tracklen = Input["Length of track (km) ", tracklen / 1000] *
   1000.0;
slope = Input["Track slope ", slope];
as = Input["Sin Amplitude (m) ", as];
bs = Input["Cos Ampliude (m )", bs];
lambda = Input["Periodic wavelength (km) ", lambda / 1000] *
   1000.0;
mass = Input["Rider+Bike  Mass (kg) ", mass];
area = Input["Rider Area (m^2) ", area];
cd = Input["Drag Coefficient ", cd];
cr = Input["Rolling friction coefficient ", cr];
outint = Input["Output interval (s)", outint];
```

- Now set up the force model equations

In[92]:=
```
theta[xp_] :=
 ArcTan[slope + as * Cos[2 * Pi * xp / lambda] * 2 * Pi / lambda -
   bs * Sin[2 * Pi * xp / lambda] * 2 * Pi / lambda]
htrack[xp_] := slope * xp + as * Sin[2 * Pi * xp / lambda] +
   bs * Cos[2 * Pi * xp / lambda];
(* Second derivative of surface for computing
 cenripical acceleration *)
dy2dx2[xp_] := -as * Sin[2 * Pi * xp / lambda] * (2 * Pi / lambda)^2 -
   bs * Cos[2 * Pi * xp / lambda] * (2 * Pi / lambda)^2;
vmag[xd_, zd_] := Sqrt[xd^2 + zd^2];

(* Acceleration due to drag *)
dragx[xd_, zd_] := -rhoair * vmag[xd, zd] * area * cd *
   xd / (2 * mass);
dragz[xd_, zd_] := -rhoair * vmag[xd, zd] * area * cd *
   zd / (2 * mass);
(* Rolling force.  This act along surface *)
rollx[xp_] :=  -grav * cr * Cos[theta[xp]];
rollz[xp_] := -grav * cr * Sin[theta[xp]];
(* Gravity force acting perpendicular to surface *)
gravx[xp_] := -grav * Cos[theta[xp]] * Sin[theta[xp]];
gravz[xp_] := +grav * Cos[theta[xp]]^2;
(* Centripetal acceleration *)
(* Could divide by Sqrt[(1+Tan[theta[xp]]^2)^3]]
 factor to curvature term.  This change does not
 seem to make solution closer to track shape*)
```

```
centx[xp_, xd_, zd_] :=
  (xd^2 + zd^2) * Sin[theta[xp]] * dy2dx2[xp];
centz[xp_, xd_, zd_] :=
  (xd^2 + zd^2) * Cos[theta[xp]] * dy2dx2[xp];
(* This commented code has expression for radius of
  curvature
  centx[xp_,xd_,zd_] :=
 (xd^2+zd^2)*Sin[theta[xp]]*
  dy2dx2[xp]/Sqrt[(1+Tan[theta[xp]]^2)^3];
centz[xp_,xd_,zd_] :=
 (xd^2+zd^2)*Cos[theta[xp]]*
  dy2dx2[xp]/Sqrt[(1+Tan[theta[xp]]^2)^3];
*)
(* To see the effects of the centripetal force,
remove the comments below
  centx[xp_,xd_,zd_] :=0;
centz[xp_,xd_,zd_] :=0;
*)

(* Now the rider force *)
ridrx[xp_, xd_, zd_] :=
  (Min[If[ vmag[xd, zd] > 0, prider / vmag[xd, zd], fmax],
      fmax] / mass) * Cos[theta[xp]];
ridrz[xp_, xd_, zd_] :=
  (Min[If[ vmag[xd, zd] > 0, prider / vmag[xd, zd], fmax],
      fmax] / mass) * Sin[theta[xp]];
Print[Acceleration functions Set]
```

```
Acceleration functions Set
```

- The following cell can be used to test that the acceleration functions above generate numeric results when called with distance and velocity. This test is useful if NDSolve does not return answer.

In[121]:=

```
dx = 1500.; txv = 2.0; tzv = 0.01;
Print["Slope ", theta[dx], " 2nd derivative ",
 dy2dx2[dx], " Height ", htrack[dx]]
Print["Drag X ", dragx[txv, tzv], " Z ", dragz[txv, tzv]]
Print["Roll X ", rollx[dx], " Z ", rollz[dx]]
Print["Gravity X ", gravx[dx], " Z ", gravz[dx]]
Print["Centripetal X ", centx[dx, txv, tzv], " Z ",
 centz[dx, txv, tzv]]
Print["Rider X ", ridrx[dx, txv, tzv], " Z ",
 ridrz[dx, txv, tzv]]
```

```
Slope 0.001 2nd derivative 0.000049348 Height -3.5

Drag X -0.0184822 Z -0.0000924109

Roll X -0.0686 Z -0.0000686

Gravity X -0.00979999 Z 9.79999

Centripetal X 1.97397×10^-7 Z 0.000197397

Rider X 0.25 Z 0.00025
```

- Now set up the solution for NDSolve. There are two equations in the x and z accelerations and initial conditions for x and z and x' and z' at time zero. The solution is set to solve for a maximum of 10000 seconds. This is OK for the standard case but may need to be modified for other longer running cases.

  The evaluaton of the solution is saved for x and z positions and z and z velocities. (NDSolve contains examples of setting up these solutions.

  We can then find the length of the time need by solving the equation xp[t]-tracklen == 0. This is not with FindRoot.

In[177]:=

```
solution =
  NDSolve[
   {x''[t] == ridrx[x[t], x'[t], z'[t]] + dragx[x'[t], z'[t]] +
      rollx[x[t]] + gravx[x[t]] + centx[x[t], x'[t], z'[t]],
    z''[t] == ridrz[x[t], x'[t], z'[t]] + dragz[x'[t], z'[t]] +
      rollz[x[t]] + gravz[x[t]] - grav +
      centz[x[t], x'[t], z'[t]], x[0] == 0, z[0] == 0,
    x'[0] == 0, z'[0] == 0}, {x, z}, {t, 0, 10 000}];

xp[t_] := First[Evaluate[x[t] /. solution]];
zp[t_] := First[ Evaluate[z[t] /. solution]];
xv[t_] :=  First[Evaluate[x'[t] /. solution]];
zv[t_] :=  First[Evaluate[z'[t] /. solution]];
(* Compute the error in the distance *)
endt  = t /. FindRoot[xp[t] - tracklen == 0 , {t, 1, 3000.0}];

(* Now integrate to get work done *)
work = NIntegrate[mass * Sqrt[xv[t]^2 + zv[t]^2] *
    Sqrt[ridrx[xp[t], xv[t], zv[t]]^2 +
      ridrz[xp[t], xv[t], zv[t]]^2], {t, 0, endt},
   AccuracyGoal → 4];
Print["Time to reach end of track ", endt, " sec, Speed ",
 xv[endt], " m/s"];
Print["Work done ", work, " Joules, ", work / 4.1868 / 10^3,
 " kcal" ];
```

NIntegrate::ncvb :

NIntegrate failed to converge to prescribed accuracy after 9 recursive bisections in t near {t} = {678.279}. NIntegrate obtained 174398.48726962117` and 0.39321119853291564` for the integral and error estimates. ≫

```
Time to reach end of track 2255.17 sec, Speed 2.65009 m/s

Work done 174 398. Joules, 41.6544 kcal
```

- The case above gives an example of how tricky Mathematica can be in telling you what it is actually doing. In the integration above for work a numerical rounding error message is printed. The message suggests there is some problem around 678 seconds. The cell below divides the calculation into 2 parts normally split at around 678 seconds and in this case no numerical error warning is printed. It's not at all clear why the program is behaving this way.

In[166]:=

```
tt = 678.279;
worksplit =
  NIntegrate[mass * Sqrt[xv[t]^2 + zv[t]^2] *
     Sqrt[ridrx[xp[t], xv[t], zv[t]]^2 +
       ridrz[xp[t], xv[t], zv[t]]^2], {t, 0, tt}] +
   NIntegrate[mass * Sqrt[xv[t]^2 + zv[t]^2] *
     Sqrt[ridrx[xp[t], xv[t], zv[t]]^2 +
       ridrz[xp[t], xv[t], zv[t]]^2], {t, tt, endt}];
Print["Work computed in one step ", work,
  " and split at time ", tt, " ", worksplit,
  ". Difference in results ", work - worksplit];
```

```
Work computed in one step 174 399. and split at time
 678.279 174 399.. Difference in results 0.
```

- Final output for version A (full solutions)

In[185]:=

```
Print["12.010 HW 4: Mathematica Bike Problem"];
Print["Solution Parameters"];
Print["Track Length ", tracklen / 1000.0, " km"];
Print["Track Slope ", slope, " Sin Cos ", as, " ",
  bs, " m, Lambda ", lambda / 1000., " km"];
Print["Rider Power ", prider, " Watts, Max Force ",
  fmax, " N"];
Print["Time to reach end of track ", endt, " sec, Speed ",
 xv[endt], " m/s"]
Print["Work done by rider ", work, " Joules, ",
 work / 4.1868 / 10^3, " kcal" ]
```

```
12.010 HW 4: Mathematica Bike Problem

Solution Parameters

Track Length 10. km

Track Slope 0.001 Sin Cos 5. 0. m, Lambda 2. km

Rider Power 100 Watts, Max Force 20 N

Time to reach end of track 2255.17 sec, Speed 2.65009 m/s

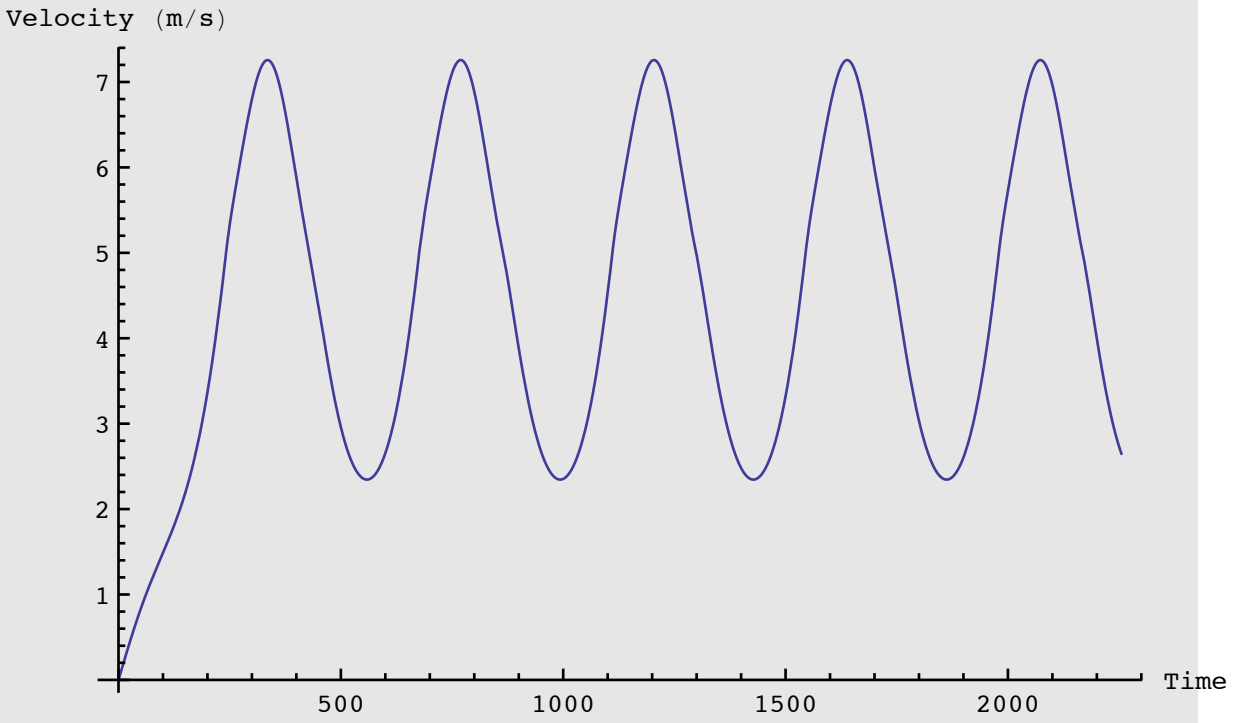
Work done by rider 174 398. Joules, 41.6544 kcal
```

- Add some graphics to the results: Velocity versus time, Z position versus X position, and difference from track shape. The latter plot shows how well our integration matched the actual shape of the track.

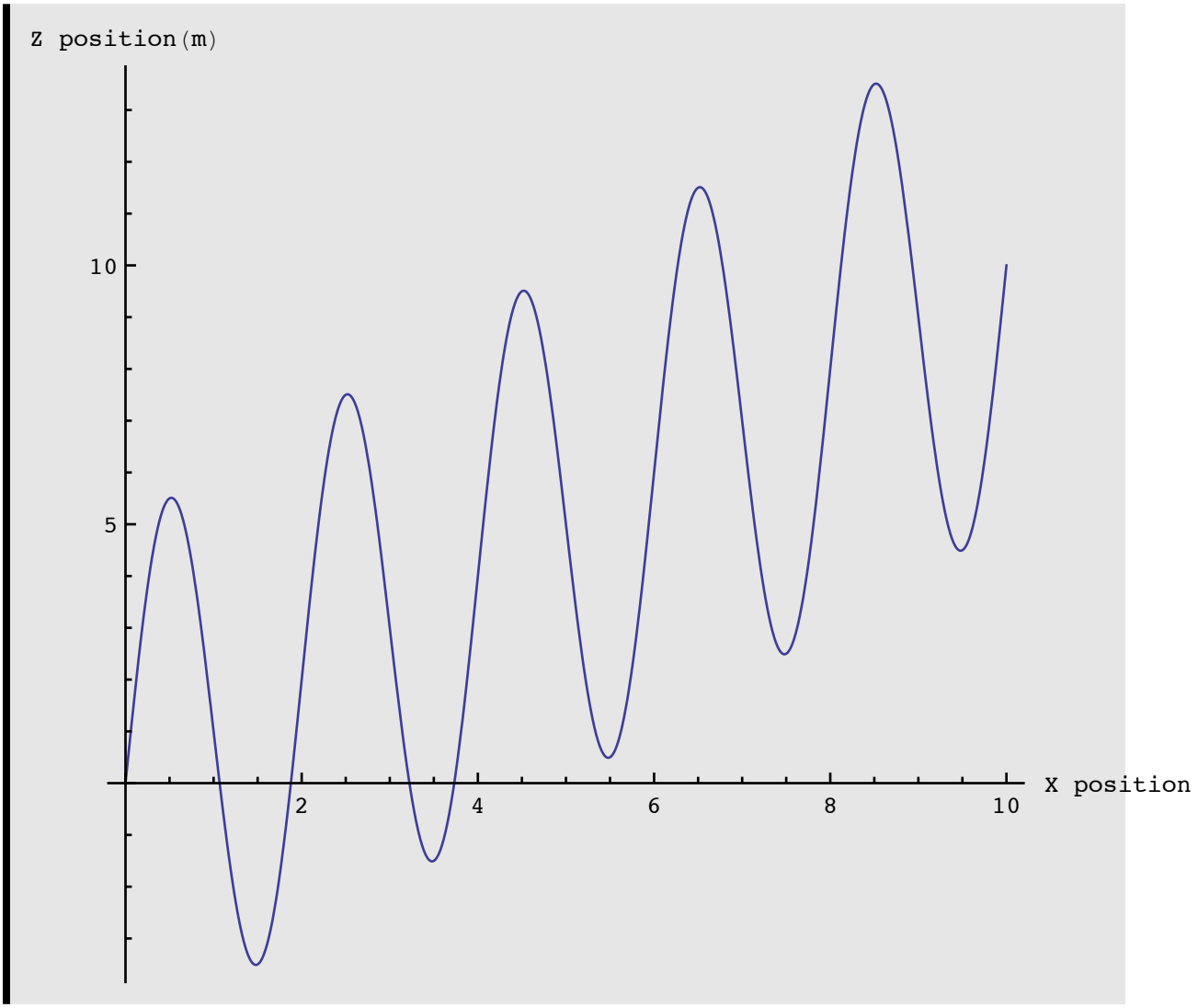
In[241]:=

```
Plot[ xv[t], {t, 0, endt},
 AxesLabel → {"Time (sec)", "Velocity (m/s)"}]
ParametricPlot[ {xp[t] / 1000 , zp[t]}, {t, 0, endt},
 AxesLabel → {"X position (km)", "Z position(m)"},
 AspectRatio → 1 / 1]
ParametricPlot[ {xp[t] / 1000 , (zp[t] - htrack[xp[t]]) * 1000.},
 {t, 0, endt},
 AxesLabel → {"X position (km)", "Error in Z (mm)"},
 AspectRatio → 1 / 1]
```

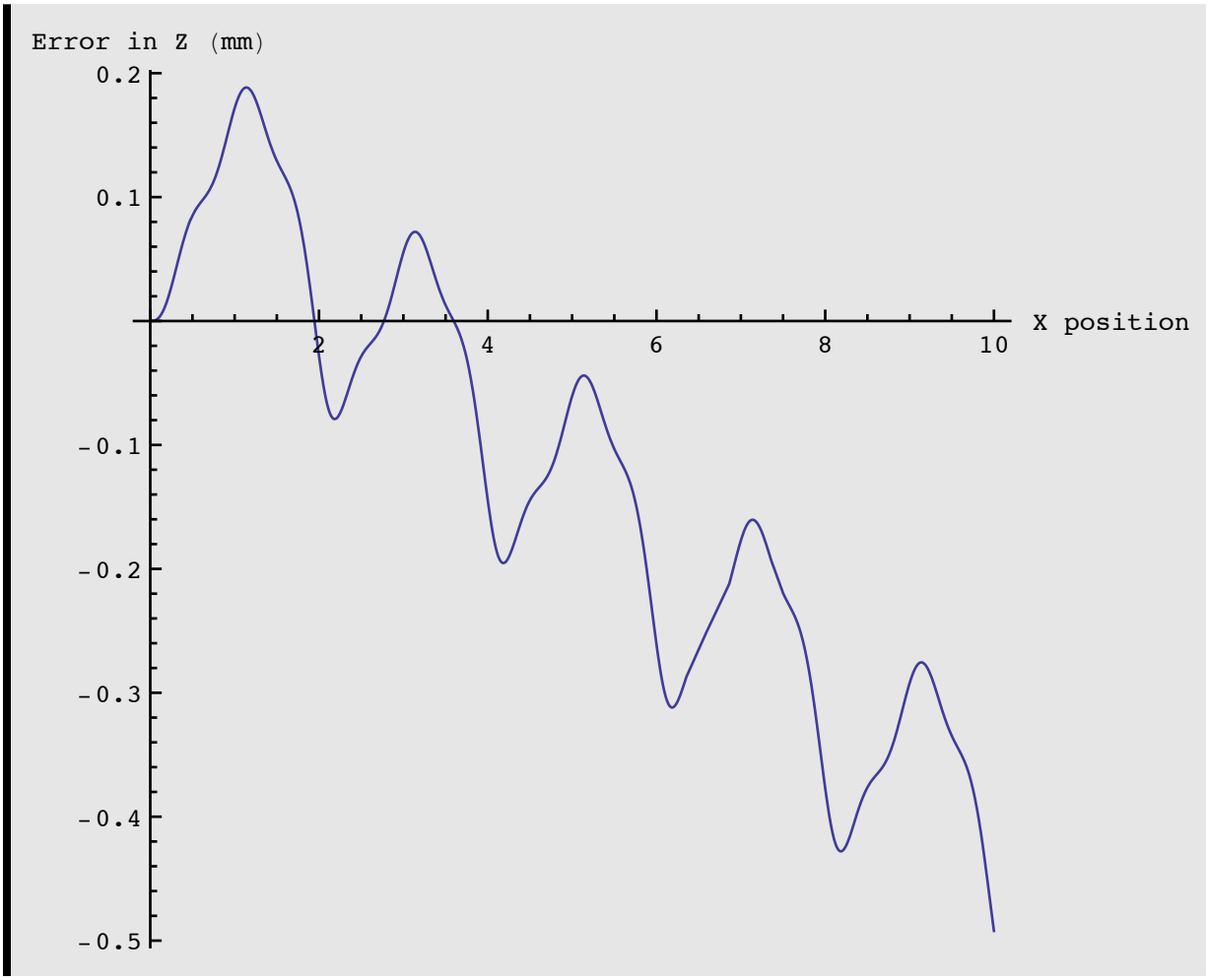
Out[241]=



Out[242]=



Out[243]=



- We now use table and table form to have to put the positions and velocities as a function of the interval selected by the user. (The default interval is 100 seconds). To generate the table with 100s output, and to add the final value, we generate the table in two parts and save the table in a list called outlst. We use the append function to join the two tables together.

In[274]:=
```
outlst = Table[{t, xp[t], zp[t], xv[t], zv[t]},
    {t, 0, endt, outint}];
(* Now add the final entry to the tabe; *)
outlst =
  Append[outlst,
   Transpose[Table[{t, xp[t], zp[t], xv[t], zv[t]},
     {t, endt, endt, outint}]]];
TableForm[outlst,
 TableHeadings →
  {None, {"Time (s)", "X Pos (m)", "Z pos (m)",
    "X Vel (m/s)", "Z Vel (m/s)"}}]
```

Out[276]//TableForm=

Time (s)	X Pos (m)	Z pos (m)	X Vel (m/s)	Z Vel (m/s)
0	0.	0.	0.	-5.29396×10^{-23}
100	79.9901	1.32329	1.48869	0.0241386
200	306.763	4.41341	3.36696	0.0335387
300	826.797	3.41531	6.79761	-0.0845564
400	1513.81	-3.48135	5.87698	0.00988077
500	1951.68	1.19559	2.95947	0.0489105
600	2202.42	5.17194	2.65422	0.0361969
700	2608.52	7.32075	5.8297	-0.0247871
800	3292.9	-0.685554	6.88819	-0.0586485
900	3834.83	1.35511	3.88251	0.0568391
1000	4117.37	5.91931	2.35353	0.0368374
1100	4428.12	9.30101	4.53331	0.0204786
1200	5046.56	4.31778	7.24973	-0.105412
1300	5681.7	1.47426	4.97372	0.0471862
1400	6035.25	6.58758	2.48085	0.0412101
1500	6294.17	10.2845	3.32052	0.0347469
1600	6803.7	9.69517	6.71443	-0.0793262
1700	7493.54	2.49435	5.98646	0.00407604
1800	7941.44	7.0264	3.03341	0.0498764
1900	8193.43	11.0479	2.60451	0.0361917
2000	8588.77	13.3952	5.72489	-0.0190286
2100	9269.22	5.52648	6.96317	-0.0655712
2200	9821.37	7.16001	3.99206	0.0570797
2255.17	10 000.	9.99951	2.65009	0.0442763

- **Alternative solution,** Here we solve the one dimensional problem which is basically the roller coaster solution that keeps the bike on the ground. In the Fortran and C-versions we computed the forces along the sloped surface and integrated horizontal motions from the motion along the slope. If a similar approach is followed here, then we need expressions for height and slope as functions of the dis-

tance along the sirface. These could be derived given that we have the eqautions. A simpler solution to solve for the horizontal motion and compute the z-motion consistent wth staying on the track. The z-motion is needed because drag and rider force depend on the total velocity not just the horizontal velocity.

We use the same constants above but we re-define the accerations and NDSolve here.

In[284]:=

```
theta[xp_] :=
 ArcTan[slope + as * Cos[2 * Pi * xp / lambda] * 2 * Pi / lambda -
   bs * Sin[2 * Pi * xp / lambda] * 2 * Pi / lambda]

(* Get total velocity given running on track *)
vtot[xd_, xp_] := Sqrt[xd^2 + (xd * Tan[theta[xp]])^2];

dacc[xd_, xp_] := -cd * rhoair * vtot[xd, xp]^2 *
   area / (2 * mass);
racc[th_] := -grav * cr * Cos[th];
facc[xd_, xp_] :=
 Min[If[ vtot[xd, xp] > 0, prider / vtot[xd, xp], fmax], fmax] /
  mass
gacc[th_] := -grav * Sin[th];
Print["1-D accelerations set"]
```

```
1-D accelerations set
```

```
(* Set up differential equations to be be solved,
x[t] is horizontal position,
There is still a problem here with along track versus
 horizontal distance
 Here we use y for the dependent variable*)
solnb =
  NDSolve[
   {y''[t] ==
     (dacc[y'[t], y[t]] + gacc[theta[y[t]]] +
        racc[theta[y[t]]] + facc[y'[t], y[t]]) *
      Cos[theta[y[t]]],
    y[0] == 0, y'[0] == 0.0}, y, {t, 0.0, 10000.0},
   AccuracyGoal → 10];
py[t_] := Evaluate[y[t] /. solnb];
vy[t_] := Evaluate[y'[t] /. solnb];
(* Compute the error in the distance *)
endb = t /. FindRoot[py[t] - tracklen == 0 , {t, 1, 3000.0}];
work = NIntegrate[mass * facc[First[vy[t]], First[py[t]]] *
    vtot[First[vy[t]], First[py[t]]], {t, 0, endt}];
Print["Time to reach end of track ", endb, " sec, Speed ",
  First[vy[endt]], " m/s"];
Print["Work done ", work, " Joules, ", work / 4.1868 / 10^3,
  " kcal" ];
```

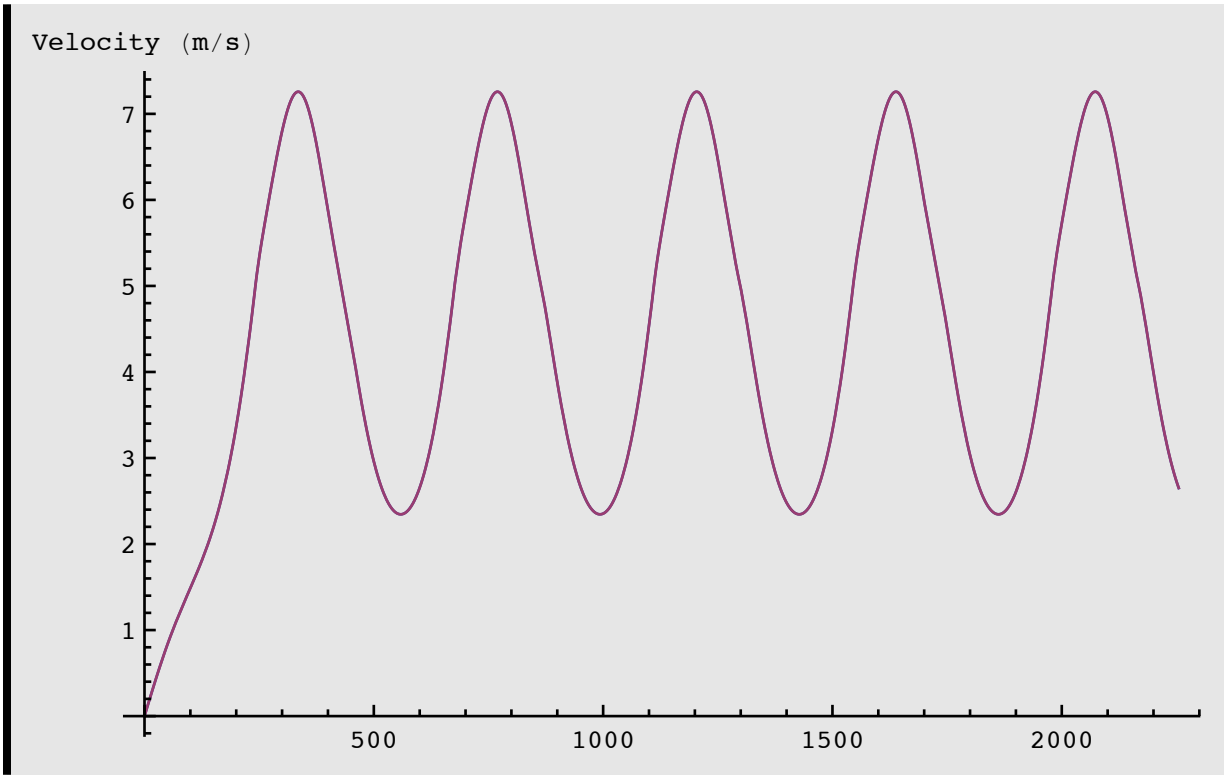
```
Time to reach end of track 2255.05 sec, Speed 2.64848 m/s

Work done 174405. Joules, 41.6558 kcal
```

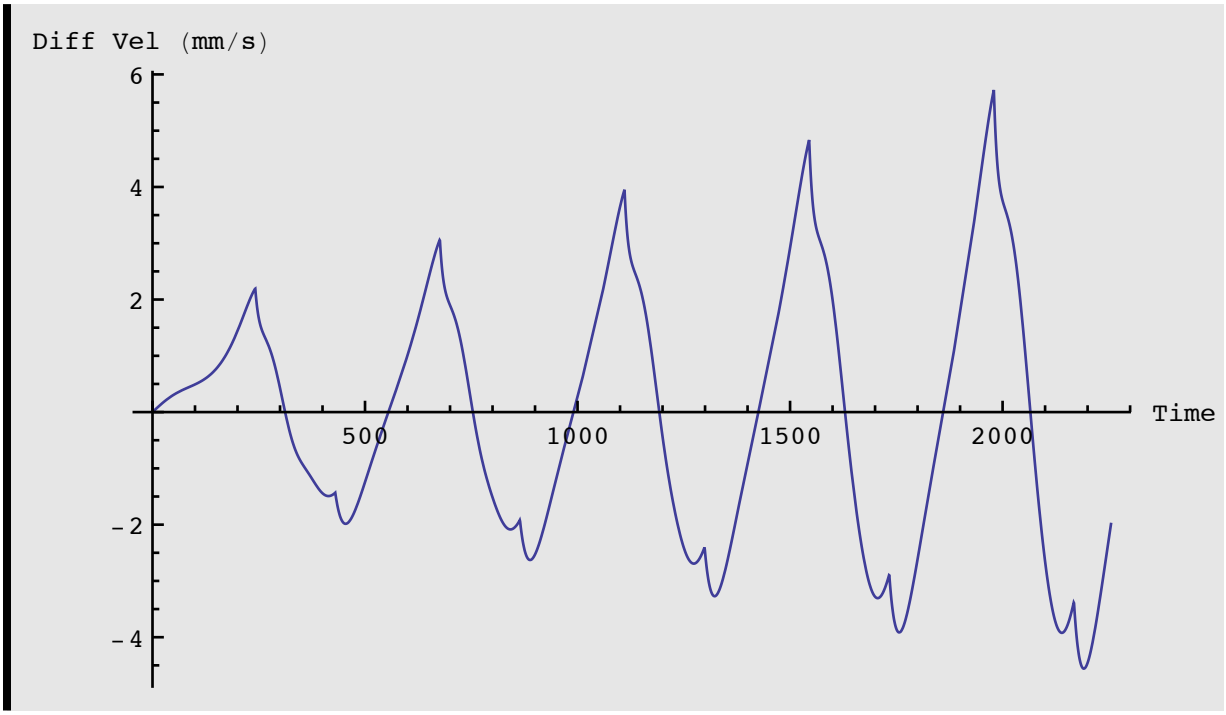
In[311]:=

```
Plot[{vy[t], Sqrt[xv[t]^2 + zv[t]^2]}, {t, 0, endb},
 PlotRange → All,
 AxesLabel → {"Time (sec)", "Velocity (m/s)"}]
Plot[(vy[t] - Sqrt[xv[t]^2 + zv[t]^2]) * 1000, {t, 0, endb},
 PlotRange → All,
 AxesLabel → {"Time (sec)", "Diff Vel (mm/s)"}]
```

Out[311]=



Out[312]=



- Now output table of values

In[308]:=

```
outl1d = Table[{t, py[t], vy[t]}, {t, 0, endb, outint}];
(* Now add the final entry to the tabe; *)
outl1d =
  Append[outl1d,
   Transpose[Table[{t, py[t], vy[t]},
     {t, endb, endb, outint}]]];
TableForm[outl1d,
 TableHeadings →
  {None, {"Time (s)", "X Pos (m)", "X Vel (m/s)"}}]
```

Out[310]//TableForm=

Time (s)	X Pos (m)	X Vel (m/s)
0	0.	0.
100	80.0296	1.48938
200	306.907	3.36857
300	827.102	6.79856
400	1514.08	5.87554
500	1951.82	2.95863
600	2202.58	2.65548
700	2608.89	5.83165
800	3293.34	6.88692
900	3835.08	3.8804
1000	4117.54	2.35409
1100	4428.5	4.537
1200	5047.16	7.25006
1300	5682.12	4.97145
1400	6035.47	2.48028
1500	6294.5	3.32362
1600	6804.39	6.7169
1700	7494.16	5.98318
1800	7941.75	3.03121
1900	8193.74	2.6065
2000	8589.47	5.72868
2100	9270.08	6.96082
2200	9821.86	3.98801
2255.05	10 000.	2.65031

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #5 **Due Thursday, December 01, 2011**

Question (1): (25-points) (a) Write a Matlab M-file which generates a table of error function (erf) and its derivatives for real arguments (z) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the M-file and give an example of the output.
(b) How would you change this M-file if 10 significant digits were required?
Matlab M-file should also be supplied

Question (2): (25-points).
Write an M-file that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The M-file should not be specific to the lengths of your name (ie., the M-file should work with anyone's name.
As an example. An input of
thomas abram herring
would generate:
Herring, Thomas A.

Question (3): (50-points) Write a Matlab M-file that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where *H(x)* is the height of the path above the starting height, *S* is a slope in m/m, *A* and *B* are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your M-file can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The M-file can assume a constant power scenario and constant force at low velocities.

As a test of your M-file use the following constants to compute:

(a) Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.
(b) The position and velocity of the bike tabulated at a 100-second interval.
(c) Add graphics to your M-file which plots the velocity of the bike as a function of time and position along the path.

Assume the following values

C_d = 0.9

C_r = 0.007

A_r = 0.67 m^2
r = 1.226 km/m^3
g = 9.8 m/s^2

M_r = 80 kg

In this case, the Matlab M-file will not be of the type used for fortran and C/C++. Look at the documentation on ODExx where xx is a pair of number for Ordinary Differential Equation solutions.
Your answer to this question should include:

(a) The algorithms used and the design of your M-file
(b) The Matlab M-file with your code and solution (I run your M-file).
(c) The results from the test case above.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

12.010 Homework #5 Solution **Due Thursday, December 01, 2011**

Question (1): *(25-points) (a) Write a Matlab M-file which generates a table of error function (erf) and its derivatives for real arguments (z) between -3 and 3 in steps of 0.25. The error function is defined by the equation below (but is rarely evaluated by performing the integration).*

$$\operatorname{erf}(z) \equiv \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt.$$

(see http://mathworld.wolfram.com/Erf.html for information the error function)
The values in the table should be given with 5 decimal places. The table should have headers explaining what the columns are. Explain how you designed the M-file and give an example of the output.
(b) How would you change this M-file if 10 significant digits were required?
Matlab M-file should also be supplied

Solution:

The solution to this problem is easy in Matlab because the Erf function is built in. In the solution we use Matlab's vector operations to form the entries for the table in one single line. By using the correct transpose on the results we are able to print the table with one print statement. We also added to the solution plots of the function and its derivative.
The solution is available at
http://geoweb.mit.edu/~tah/12.010/HW05_01_2011.m

Below is shown in the output from the program for 5 significant digits. To increase the number of significant digits, the format for the table simply needs to be changed. All of the calculations were done with double precision and therefore no change to the variables needs to be made.

```
12.010 HW05 Q01: Table of Erf and its derivative
------------------------------------
|  Arg x  |    Erf(x)   |  d(Erf)/dx |
------------------------------------
|  -3.000 |   -0.99998  |    0.00014 |
|  -2.750 |   -0.99990  |    0.00059 |
|  -2.500 |   -0.99959  |    0.00218 |
|  -2.250 |   -0.99854  |    0.00714 |
|  -2.000 |   -0.99532  |    0.02067 |
|  -1.750 |   -0.98667  |    0.05277 |
|  -1.500 |   -0.96611  |    0.11893 |
|  -1.250 |   -0.92290  |    0.23652 |
|  -1.000 |   -0.84270  |    0.41511 |
|  -0.750 |   -0.71116  |    0.64293 |
|  -0.500 |   -0.52050  |    0.87878 |
```

-0.250	-0.27633	1.06001
0.000	0.00000	1.12838
0.250	0.27633	1.06001
0.500	0.52050	0.87878
0.750	0.71116	0.64293
1.000	0.84270	0.41511
1.250	0.92290	0.23652
1.500	0.96611	0.11893
1.750	0.98667	0.05277
2.000	0.99532	0.02067
2.250	0.99854	0.00714
2.500	0.99959	0.00218
2.750	0.99990	0.00059
3.000	0.99998	0.00014

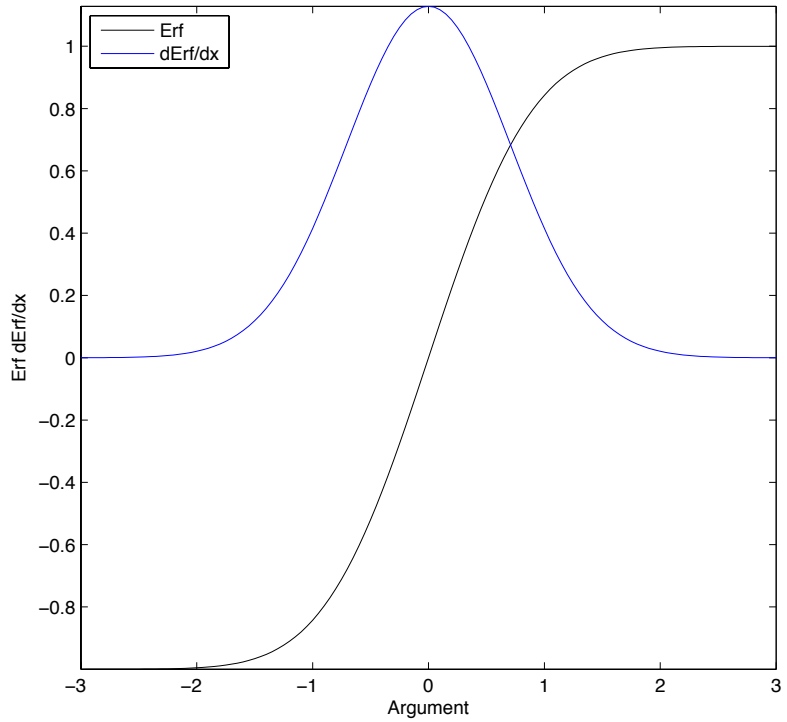


Figure 1. Plot of the Erf function and its derivative.

Question (2): *(25-points).*

Write an M-file that reads your name in the form <first name> <middle name> <last name> and outputs the last name first and adds a comma after the name, the first name, and initial of your middle name with a period after the middle initial. If the names start with lower case letters, then these should be capitalized. The M-file should not be specific to the lengths of your name (ie., the M-file should work with anyone's name. As an example. An input of

thomas abram herring
would generate:
Herring, Thomas A.

Solution:

This solution is also easy in Matlab using the tolower and toupper functions. The code also detects whether two or three names have been given and changes the output accordingly.

Solution is available at
http://geoweb.mit.edu/~tah/12.010/HW05_02_2011.m

```
---------------------------------------
12.010 HW 05 Q2:
Enter Name (first middle last) thomas abram herring
Herring, Thomas A.
```

Question (3): *(50-points) Write a Matlab M-file that will compute the motion of a bicyclist and the energy used cycling along an oscillating, sloped straight-line path. The path followed will be expressed as*

$$H(x) = Sx + A\sin(2\pi x/\lambda) + B\cos(2\pi x/\lambda)$$

where H(x) is the height of the path above the starting height, S is a slope in m/m, A and B are amplitudes of sinusoidal oscillations in the path. The wavelength of the oscillations is λ. The forces acting on the bicycle are:

$$\text{Wind Drag} \quad F_d = 1/2 A_r C_d \rho V^2$$

$$\text{Rolling Drag} \quad F_r = M_r g C_r$$

where A_r is the cross-sectional area of the rider, C_d is the drag coefficient, r is the density of air and V is the velocity of the bike. For the rolling drag, M_r is the mass of the rider and bike, g is gravitation acceleration and C_r is rolling drag coefficient.

The bicyclist puts power into the bike by pedaling. The force generated by this power is given by

$$\text{Rider force} \quad F_r = P_r / V$$

where F_r is the force produced by the rider, P_r is power used by the rider and V is velocity that the bike is traveling (the force is assumed to act along the velocity vector of the bike). Your M-file can assume that the power can be used at different rates along the path. The energy used will be the integrated power supplied by the rider. Assume that there is maximum value to the rider force.

Your code should allow for input of the constants above (path and force coefficients). The M-file can assume a constant power scenario and constant force at low velocities.

As a test of your M-file use the following constants to compute:

(a) *Time to travel and energy used to travel 10 km along a path specified by S=0.001, A=5.0 m, B=0.0 m and λ= 2km, with constant power use of P_r =100Watts and a maximum force available of 20N.*
(b) *The position and velocity of the bike tabulated at a 100-second interval.*
(c) *Add graphics to your M-file which plots the velocity of the bike as a function of time and position along the path.*

Assume the following values

$C_d = 0.9$

$C_r = 0.007$

$A_r = 0.67\ m^2$

$\rho = 1.226\ kg/m^3$

$g = 9.8\ m/s^2$

$M_r = 80\ kg$

In this case, the Matlab M-file will not be of the type used for fortran and C/C++. Look at the documentation on ODExx where xx is a pair of number for Ordinary Differential Equation solutions.

Your answer to this question should include:

(a) *The algorithms used and the design of your M-file*
(b) *The Matlab M-file with your code and solution (I run your M-file).*
(c) *The results from the test case above.*

Solution:

The solution to this problem is also quite easy in Matlab. We use the OED 45 1st order differential equation solver. To use this function we also have a function that computes the acceleration at the bike and another function that is the event detector that determines when the bike reaches 10 km distance. For ease of coding, the variables that control the characteristics of the bike are saved as globals. We also use an input dialog box to both set the defaults and allow the user to change the values at runtime.
Also included in the solution is an animation and plot of the motion of the bike. To have the animation runs smoothly we change the output interval of the differential equation solver so that we generate more points then those that given in the standard 100 seconds separated values. There are some interesting bugs in Matlab for plotting the velocity vectors. The quiver option is used for this plot but in the current release of Matlab the arrowheads are extremely large and are turned off in the plots. Also the automatic scaling of the vectors does not generate reasonable length vectors, and so in these plots we manually set the scale factor.

To compute the energy used during a bike ride, we simply added the derivative of the energy to the differential equations. This way the total energy was integrated automatically in the solution and could be output to the table as shown below.

The solution M-files are available at:
http://geoweb.mit.edu/~tah/12.010/HW05_03_2011.m
http://geoweb.mit.edu/~tah/12.010/bikeacc.m
http://geoweb.mit.edu/~tah/12.010/bikehit.m
http://geoweb.mit.edu/~tah/12.010/banimate.m

The table and figures below show the results for the default solution.

```
------------------------------------------------------------------------
12.010 HW 05 Q 3: Bike simulation
------------------------------------------------------------------------
    Time          X pos       Z pos       X Vel       Z Vel       Energy
   (sec)            (m)         (m)       (m/s)       (m/s)     (Joules)
     0.000        0.0000      0.0000      0.0000      0.0000         0.00
   100.000       79.9607      1.3228      1.4894      0.0242      1599.43
   200.000      306.7804      4.4136      3.3662      0.0335      6136.26
   300.000      826.7200      3.4173      6.7979     -0.0845     15406.88
   400.000     1513.7483     -3.4812      5.8778      0.0099     25406.88
   500.000     1951.5589      1.1930      2.9595      0.0489     33908.48
   600.000     2202.3092      5.1692      2.6551      0.0362     38924.12
   700.000     2608.2191      7.3210      5.8263     -0.0247     46828.92
   800.000     3292.5203     -0.6831      6.8901     -0.0588     56828.92
   900.000     3834.4703      1.3482      3.8829      0.0568     66438.93
  1000.000     4117.1190      5.9128      2.3536      0.0368     72092.64
  1100.000     4427.6762      9.2952      4.5307      0.0206     78304.19
  1200.000     5045.7397      4.3271      7.2502     -0.1055     88249.44
  1300.000     5681.1563      1.4650      4.9724      0.0471     98248.16
  1400.000     6034.8791      6.5772      2.4814      0.0412    105321.74
  1500.000     6293.6398     10.2743      3.3172      0.0348    110497.49
  1600.000     6802.7335      9.7025      6.7096     -0.0791    119675.95
  1700.000     7492.6695      2.4890      5.9908      0.0038    129675.95
  1800.000     7941.2097      7.0155      3.0363      0.0499    138331.57
  1900.000     8193.2972     11.0363      2.6051      0.0362    143373.96
  2000.000     8588.5246     13.3858      5.7243     -0.0190    151121.55
  2100.000     9268.9477      5.5189      6.9633     -0.0657    161121.55
  2200.000     9821.1684      7.1456      3.9935      0.0571    170809.56
  2255.202    10000.0000      9.9890      2.6506      0.0443    174386.65
------------------------------------------------------------------------
Time : 2255.202, Velocity 2.6506 0.0443
Energy used:    174386.65 Joules,    41.64 kcals

To show animation: banimate(ta,ya)
>> banimate(ta,ya)
```

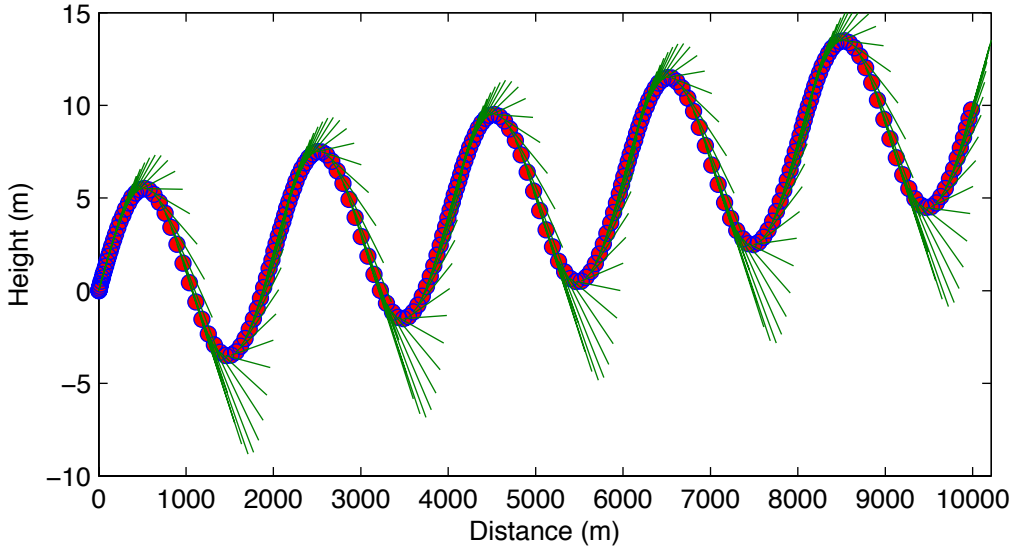


Figure 2: Trajectory of the bike along with vectors showing its velocity (green lines). Due to poor scaling in Matlab, no arrowheads are shown.

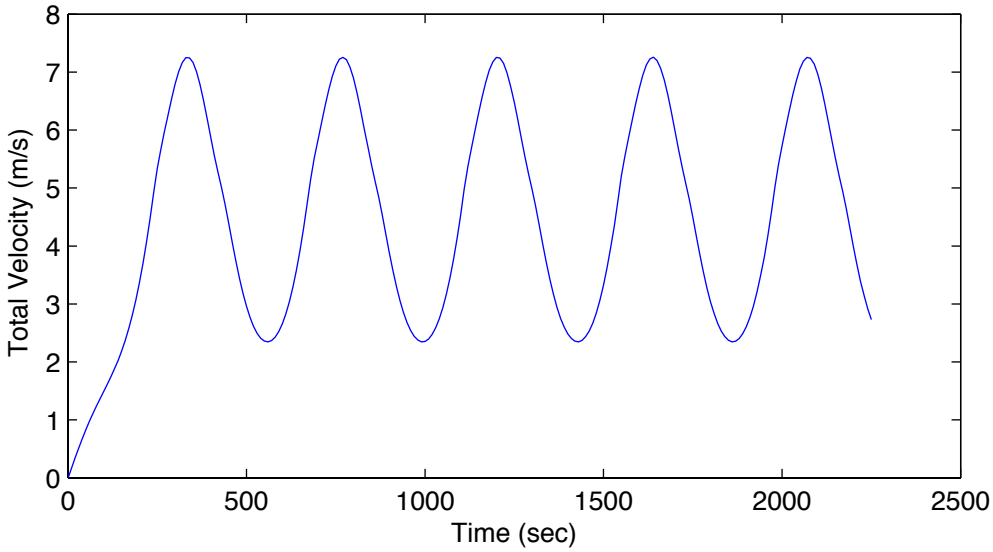


Figure 3: Velocity as a function of time for the bike.

MIT OpenCourseWare
http://ocw.mit.edu

12.010 Computational Methods of Scientific Programming
Fall 2011

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.